Master's Thesis

Graph Neural Networks as Triggers for Ocean-Based Neutrino Telescopes

Department of Physics
Ludwig Maximilian University
Munich

Avalon Rego

Munich, April 1, 2025

Submitted in partial fulfilment of the requirements for the degree of M. Sc.

Externally supervised by Dr. Francesca Capel
Internally supervised by Prof. Dr. Otmar Biebel

Masterarbeit

Graphen-Neuronale Netzwerke als Trigger für Neutrinoteleskope im Ozean

Fakultät für Physik
Ludwig-Maximilians-Universität
München

Avalon Rego

München, April 1, 2025

Extern betreut von Dr. Francesca Capel
Intern betreut von Prof. Dr. Otmar Biebel

Declaration

I hereby declare that this thesis is my own work, and that I have not used any sources and aids other than those stated in the thesis.

Munich, April 1, 2025

Avalon Rego

Acknowledgements

My deepest gratitude goes to Dr. Francesca Capel, whose patience, encouragement, and expertise have made this journey both rewarding and enriching.

I would also like to thank Ananya Shankar for various insightful discussions and for submitting this thesis on my behalf.

I would also like to thank all my friends and family for their invaluable support. Your presence in my life has been a source of joy and motivation.

Abstract

Neutrinos are a window into a deeper understanding of both beyond standard model physics and various high-energy astrophysical phenomena. This is because they can easily escape dense environments due to their weakly interacting nature and can pinpoint their sources since they are not deflected by magnetic fields. We detect these weakly interacting particles by embedding detectors into massive volumes of naturally available water or ice and then detecting the Cherenkov radiation produced by their interactions. These detectors are sensitive to complex backgrounds such as bioluminescence signals which are a challenge for standard trigger algorithms. In this work, we investigate the use of Graphnet, a GNN-based Python framework, for signal classification and improving discrimination for bioluminescence signals in particular by comparing it to a standard coincidence trigger. We also explore the possibility of using this trigger to lower the energy threshold for neutrino detection.

Contents

List of Figures

List of Tables

1 Introduction

The purpose of this thesis is to develop a Graph Neural Network (GNN) trigger for the separation of neutrino event from background noise in ocean-based neutrino telescopes and to lower the hit threshold for neutrinos detection while maintaining signal purity. Many terms have been introduced without sufficient explanation, so lets clarify out terms before we move along.

To start with ocean-based neutrino telescopes, neutrinos are not directly visible but rather are only visible through secondary particles they have interacted with. These secondary particles then need to be energetic enough to produce Cherenkov radiation, which can then be detected by us. Cherenkov radiation is characterized by the medium in which the secondary particle is travelling [1], and due to the nature of neutrinos, we need large volumes of this medium to see significant amounts of neutrinos. Hence, neutrino telescopes take advantage of the naturally abundant quantities of water available on the surface of the Earth and use it as a detector medium. This, however, causes them to have to deal with bioluminescence, a noise source no other neutrino detector has, and is fairly complicated to deal with.

The next concept to tackle is the trigger. A trigger is a mechanism used in many Particle Physics detectors to prevent data collection until a physically interesting event comes along and satisfies certain criteria at which point the detector starts recording the data i.e. gets triggered. In this context, anything not of specific interest to us is classified as noise. Sources of noise may include the environment, the detector itself or even events going in the wrong direction may be treated as noise [2].

It is worthwhile to mention that keeping all the data form particle detectors is not really an option since they are extremely sensitive objects, where ATLAS at CERN produce raw data at a rate greater than 60 Tbps while a detector like KM3NeT produces data at a rate of ∼25 Gbps [3, 4] but with an operating time of years holding on to uninteresting data is not only pointless but also expensive. So triggers are a way to bring the data down to a manageable scale and to only have data that might be interesting for further study.

Another concept to clarify is regarding the hit threshold. Hits are the photons that hit our detector that either originated due to noise or ideally from a neutrino interaction. The number of hits of a neutrino event are correlated with energy of the neutrino, high energy events are easily distinguishable as they produce more hits. On the other hand low energy neutrinos produce far fewer hits making them hard to detect. Many triggers are based on the number of hits this creates a bias for higher energy events to get detected while low energy events get filtered out. This significantly slows down data collection as high energy neutrinos are far less abundant than the low energy neutrinos [5] that are being filtered out. Hence our aim to lower the hit threshold which really acts as a surrogate for energy. The aim is to get better statistics on the physics we are studying by having more high quality data.

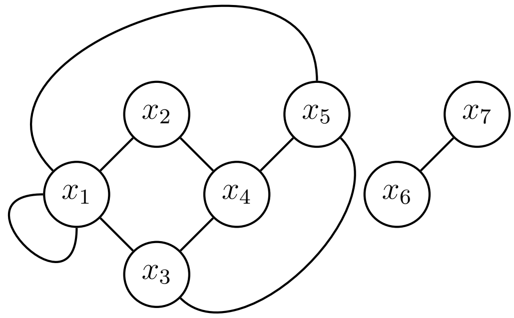


Figure 1.1: An example of a graph.

Finally, let's talk about Graph Neural Networks. Neural networks are algorithms that are capable of learning from the input you give them to perform tasks like regression, pattern recognition and for our purposes classification. Graphs here refer to nodes and edges where each node represents something or someone and the edges represent the connections/relationships between them. Putting these two ideas together Graph Neural Networks are algorithms that take in Graphs as input and can capture the relationships between the nodes and using this information perform some task we might have. The use of Graphs is particularly useful for this due to the structure of the Detectors we are considering.

This brings us to the next order of business understanding the structure of the thesis. In Chapter 2 we will go through the fascinating physics of neutrinos and how neutrino telescopes will aid in our understanding of the various mysteries that they will help uncover. In Chapter 3 we will discuss the construction and operation of neutrino telescopes. Then in Chapter 4 we discuss the simulation tools that we will use to train our GNN trigger while in Chapter 5 we discuss how we will generate data and characterise the key features of our data. In Chapter 6 we will implement a standard trigger to serve as a baseline to compare our GNN trigger. In Chapter 7 we then understand a broad overview of the basic topics associated with GNNs and the tools we will use and in Chapter 8 we finally implement our GNN trigger that then leads us to understand its performance using standard metrics and then in Chapter 9 we compare the performance of our baseline trigger to our GNN trigger and to end it all in Chapter 10 we summarise the work that was done and look at possible improvements and extensions to our work along with practical applications. So without any further delays let's begin!

2 Physics background

2.1 Neutrino Astronomy

Until very recently the only way for us to observe the universe has been with the help of the humble photon [6, 7] and while it has profoundly shaped our understanding of the universe it can't shed light on everything. For example, light is unable to provide information on extremely dense environments such as star cores, supernovae, and various other astrophysically interesting events [8–10]. There is also the issue of high-energy photons (>10 TeV) that start to interact with the background radiation at distances of around 100 Mpc effectively making the universe opaque under these conditions [11, 12]. So to work around these issues the era of multi-messenger astronomy was ushered in i.e. along with light we started using gravitational waves and more importantly for our purposes neutrinos [13, 14].

Neutrinos succeed in exactly the areas that light as a messenger fails. This is mainly due to the way it interacts with matter or more accurately its hesitance to interact with matter. Neutrinos interact exclusively through the weak force and as the name suggests they interact with everything very weakly, allowing them to escape extremely dense environments not only unaffected by matter but also much earlier than light, this lets us know that a potentially interesting astrophysical event might be occurring even before the light from the event had time to reach us allowing us to be ready to study it from start to end [15, 16]. Also due to it being a neutral particle, it is not deflected by magnetic fields allowing us to pinpoint exactly where the neutrino source was. Another perk of its weakly interacting nature is that even extremely energetic neutrinos are capable of travelling much further than light, allowing us to probe higher energies much further than what light would allow us to do. [17]

2.2 Current Research

Standard Model Extensions

The Standard model or more accurately the minimal standard model is a fantastically accurate description of the universe agreeing with experiment to an astounding degree so it is only to be expected that any place it falls short will turn into an area of intense research. Within the minimal standard model, neutrinos are massless. However, from observations of the solar neutrino flux, it was found that neutrinos oscillate into different flavours of itself and this means that neutrinos must have a mass and this leads to a host of different consequences [18, 19].

To understand a little bit about why this is such a profoundly interesting problem a little background knowledge is important. The Standard Model is a chiral theory, which means that spin $\frac{1}{2}$ particles are either left or right-handed. But to this date, we have not seen a right-handed neutrino. This is allowed in the theory but this combined with all the observations at the time made us expect a massless neutrino. The Minimal Standard Model does indeed explicitly treat it as such since in theory we could add a mass term to the Lagrangian but that would force us to make assumptions about the mass [20]. Another fact of our universe is that some massive particles are capable of mixing/oscillating into different particles from their family. This is because the flavour eigenstates of these particles are not the same as the mass eigenstates of these particles[21, 22]. What's crucial here is that these particles need to have mass for such a process to occur. So here comes the crux of the problem, our theory is confounded by 2 contradictory pieces of experimental fact. We have never seen a right-handed neutrino so the left-handed ones "should" be massless but we know that neutrinos oscillate in flavour [23] so they need to have mass. While there are many theories [24] on how this puzzle may be solved and many experiments that are working to that end as well, neutrino telescopes are uniquely positioned to provide more data in this regard.

Dark Matter

There is also the possibility of neutrino telescopes finding evidence for dark matter. Many of the theoretical solutions to the neutrino problem lead to predictions of particles. Because dark matter has only been detected through its mass and these proposed particles tend to be weakly interacting and massive they become prime candidates for being dark matter [25, 26]. It's also expected that data from neutrino telescopes will help constrain and rule out models for dark matter regardless of whether they are solutions to the neutrino mass problem based on astrophysical observations they will make [27, 28].

Astrophysical sources

One of the ways in which neutrino detectors are especially important is finding neutrino sources, such sources will help shine light on a variety of open problems. For example, since neutrinos are capable of travelling unimpeded from their source to our detector we will be able to identify cosmic accelerators which might shed light on what process produces the highest energy particles and what role these events have played in the evolution of the cosmos. Additionally, astrophysical neutrino sources can serve as independent laboratories for studying neutrino oscillations, providing a new window into this phenomenon beyond the reactor, solar and atmospheric neutrinos currently used [29–31]. And of course one of the most important features is that it allows us to probe energies that can't be achieved by even the most powerful detectors we have here on Earth giving us a unique window to probe fundamental physics. [32–34]

2.3 Neutrino Interactions

So while we have said a lot about the benefits of doing neutrino astronomy the same weakly interacting nature that makes it so fascinating also makes any individual neutrino essentially impossible to detect. Hence, it's fortunate for us that while individual neutrinos interacting is rare if there are enough of them, which thankfully there are, some of them will interact with other particles at some point. And it's these rare interactions that we will use to make our observations.

So now we make a small detour into the theory of Neutrino Interactions. The behaviour of neutrinos in the Standard Model is described by Equation 2.2 [35] i.e. the fermionic part of the Electro-Weak sector of the Standard Model Lagrangian.

$$\mathcal{L}_{SM} = \mathcal{L}_{QCD} + \mathcal{L}_{EW} + \mathcal{L}_{Higgs} + \mathcal{L}_{Yuk}; \tag{2.1}$$

$$\mathcal{L}_{fermion} = \overline{E}_L i\partial\!\!\!/ E_L + \overline{e}_R i\partial\!\!\!/ e_R + g(W^+_\mu J^{\mu+}_W + W^-_\mu J^{\mu-}_W + Z^0_\mu J^\mu_Z) + eA^\mu J^\mu_{EM}; \tag{2.2}$$

where

$$J^{\mu+}_W = \frac{1}{\sqrt{2}}\overline{\nu}_L\gamma^\mu e_L; \qquad J^{\mu-}_W = \frac{1}{\sqrt{2}}\overline{e}_L\gamma^\mu \nu_L; \qquad J^\mu_{EM} = -\overline{e}\gamma^\mu e,$$

$$J^\mu_Z = \frac{1}{\cos\theta_w}\left[\frac{1}{2}\overline{\nu}_L\gamma^\mu\nu_L + \overline{e}_L\gamma^\mu\left(\frac{1}{2} + \sin^2(\theta_w)\right)e_L + \overline{e}_R\gamma^\mu\sin^2(\theta_w)e_R\right].$$

To give a brief overview of the Lagrangian, the first term is the kinetic term of the left-handed electron doublet $E_L = (e^- \ \nu)^T_L$, the second term is the kinetic term for the right-handed electron*, the third term is the interaction term and describes how the W^+, W^- and Z bosons interact with the leptons. And lastly, θ_w is the weak mixing angle. Going into all the details of the physical implications of this equation is beyond the scope of this thesis so we shall go over relevant features as they come up. Neutrino interactions are divided into 2 parts, the Neutral Current Interactions and the Charged Current Interactions.

Neutral Current Interactions

The neutrinos neutral current interaction are mediated by the electrically neutral Z boson and is described by the third term in the Equation 2.2, specifically: $\mathcal{L}_{NC} = Z^0_\mu(J^\mu_Z)_\nu = Z^0_\mu\overline{\nu}_L\gamma^\mu\nu_L$. In the language of Feynman Diagrams, it leads to interaction vertex of the form Figure 2.1a. Z boson also couples with every other fermion so this allows for a whole range of interactions including the scattering interactions seen in Figure 2.1b.

(a) An example of a neutral scattering vertex

(b) A possible neutral scattering interaction

Figure 2.1: Comparison of NC interaction vertices

*Note the difference between how the left and right-handed particles are treated this is done to explicitly reflect the fact that we have not seen right-handed particles.

Charged Current Interactions

The CC iterations are mediated by the electrically charged $W^{\pm}$ bosons. Mathematically the interactions are described by Equation 2.3 and they lead to interaction vertices of the type seen in Figure 2.2a and these can lead to interactions of the type Figure 2.2b. With the the use of these diagrams, we can visualise the various kinds of possible interactions and specifically for our purposes we are interested in interactions that have leptons as final products, more specifically muons, this is because muons produce clear sharp signals as opposed to electrons which gives us a much more fuzzy signal. [36]

$$\mathcal{L}_{\mathrm{CC}} = \frac{g}{\sqrt{2}} \left(\bar{\nu}_L \gamma^{\mu} W^{+}_{\mu} e_L + \bar{e}_L \gamma^{\mu} W^{-}_{\mu} \nu_L \right). \tag{2.3}$$

(a) A possible Charged Current interaction vertex

(b) A possible Charged Current interaction

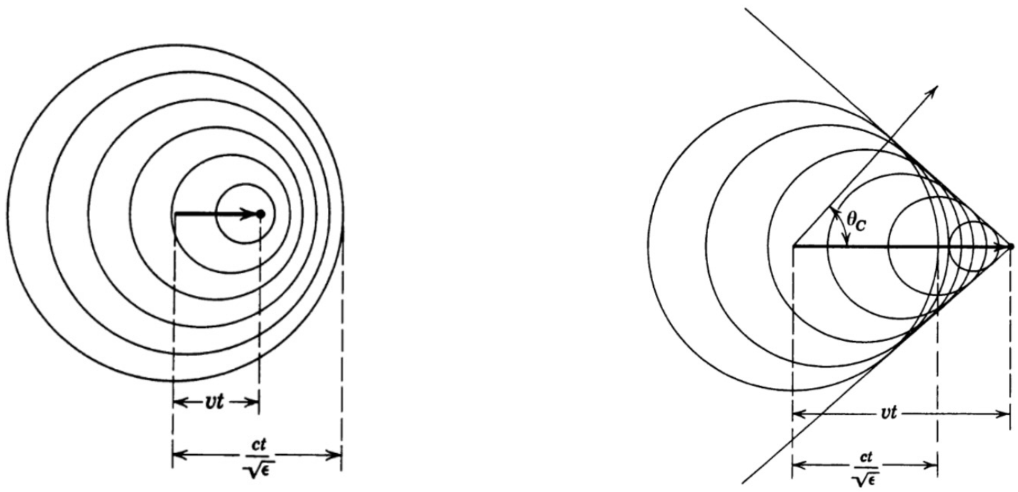
Figure 2.2: Charged current interaction vertices

2.4 Cherenkov Radiation

The Cherenkov process occurs when a charged particle moves faster than the speed of light in the medium. [36] This process is analogous to the sonic boom experienced when an object moves faster than the speed of sound in the air. Mathematically, the condition for the Cherenkov process is given by Equation 2.4 *, and when this condition is met the particle starts radiating light. This process is often used in various detectors for varying tasks. Some of the uses of this process are particle identification, path reconstruction, energy reconstruction, etc.

$$\beta > \beta_{thresh.} = \frac{1}{n} \tag{2.4}$$

*Note that the frequency dependence of all variables has been suppressed for this section.



(a) $\beta_{thresh.} > \beta$ (b) $\beta > \beta_{thresh.}$

Figure 2.3: Particle Propagation note: $n = \sqrt{\epsilon}$ [37]

Figure 2.3 shows us how the electric field of a particle propagates for the 2 cases. When $\beta_{thresh} > \beta$ the polarisation of the medium induces radiation but since it is randomly oriented it destructively interferes and we do not see any radiation. When $\beta > \beta_{thresh.}$ the induced polarisation is constructive and leads to detectable radiation.

Consider Figure 2.3b the distance covered by the particle in time t is vt while the distance covered by the electric field in the same time is ct/n. So it is easy to see that the angle of radiation θ_c is given as:

$$\theta_c = \cos^{-1}\left(\frac{1}{n\beta}\right) \tag{2.5}$$

The energy loss is given by the Frank-Tamm equation*:

$$\frac{\partial^2 E}{\partial x \partial \omega} = \frac{q^2}{4\pi}\mu\omega\left(1 - \frac{1}{\beta^2 n^2}\right) \tag{2.6}$$

These two equations form the theoretical backbone of neutrino telescopes.

2.5 Detection Principle

So based on our discussion so far we can put together that neutrino telescopes work in the following steps: A neutrino is produced somewhere in the universe ►it travels to our detector ►it interacts with our detecting medium ►this produces secondary energetic particles ►these particles then produce Cherenkov radiation ►and ideally we detect this Cherenkov light. However, due to the weak nature of neutrino interactions we need to have extremely large detectors, cubic kilometres large, so the ideal medium for our detectors is a material abundant enough to use and one of the more abundant things on plant earth is water and it's solid alter ego ice.

*The form of the equation changes if the frequency dependence of the refractive index is taken into account.

2.6 Event Signatures

Finally, we need to understand what these events can look like from a detector perspective. Depending on the interactions the end product might either be a Track or a Cascade. Tracks are produced by CC interactions that involve muon neutrinos and produce long-lived muons, due to their long-lived nature these muons then are capable of producing Cherenkov radiation along an elongated path, naturally this makes them ideal candidates to figure out what the sources of our particles are since we get a very good idea of the trajectory of the particle. High energy tracks are particularly useful since these muons are almost collinear with the muon neutrinos that produced them reducing the uncertainties on path reconstruction. [38]

Cascades on the other hand are produced by NC and CC interactions that lead to electromagnetic and hadronic showers. This means that these types of events deposit nearly all their energy into a small volume $\sim$10 m, this means due to the way detectors are spaced (see Chapter 3) only a few detecting units are capable of seeing this event. Due to their relatively small size, they are nearly symmetric and this makes it very difficult to reconstruct its path but what it lacks in path reconstruction it makes up in energy reconstruction. This is because much if not all of its energy is deposited inside a space we can observe, because of this, the task becomes relatively easy. Hence, we get excellent energy resolution [39] which is not necessarily true for tracks since it's extremely likely that they start, end or do both outside the detector making it difficult to parse out how energetic the muon was.

3 Astrophysical Neutrino Detectors

So now let us bring the ideas we discussed in Chapter 2 all together and understand what a detector that we are interested in might look like and how it functions. Experiments like these are massively complicated and to get a good understanding at each level of complexity it will be useful to break up our detector into smaller chunks (see Figure 3.1). In these chunks, along with a generic overview we will touch on specific examples from already existing or planned detectors to see how these ideas can/will be realised.

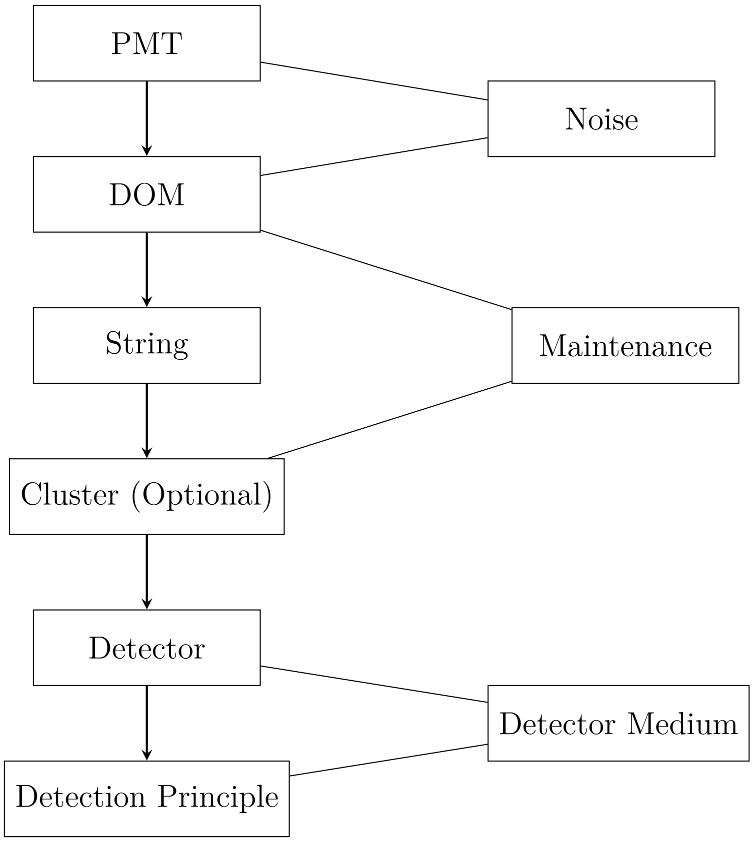
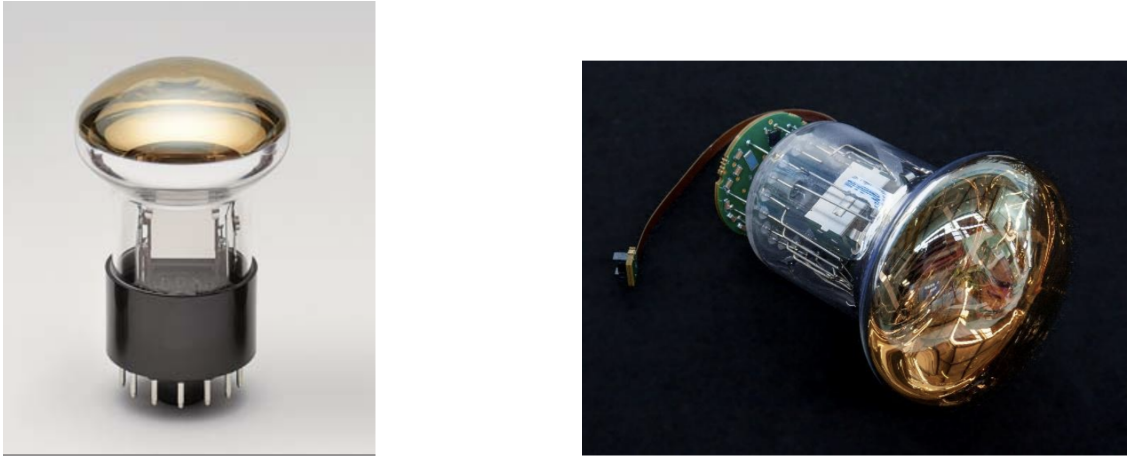


Figure 3.1: Guide to detectors

3.1 Photo-Multiplier-Tubes

Photomultiplier Tubes (PMTs) are part of the detector that does the actual detecting in detectors like IceCube, KM3Net, P-ONE, etc. These devices take advantage of the photoelectric effect to convert photons into electrical signals. However, PMTs can't perfectly convert every photon into an electrical pulse and this behaviour of the PMT is captured by the efficiency of the PMT where, for example, the peak efficiency of a PMT for KM3Net is $> 32\%$ [40]. PMTs are also capable of detecting phantom signals which we classify as noise this is generally created due to the internal working of a PMT [41] and is one of the noises that a trigger would need to filter out. However, this noise is uncorrelated and in general a more well-understood phenomenon so it is easier to deal with.



(a) Hamamatsu PMT for P-ONE [42][*]   (b) Hamamatsu PMT for KM3Net [43]

Figure 3.2: Detector PMTs

3.2 Digital Optical Modules

(a) P-ONE DOM prototype [42]   (b) KM3Net DOM [40]

Figure 3.3: Detector DOMs

A Digital Optical Module (DOM) is the container for our PMTs, they contain anywhere between $10-31$ PMTs[†] the construction of these are extremely important for the life cycle of the detectors, especially sea-based detectors since, unlike IceCube which only needs to deal with a solid i.e. ice, water-based detectors need to be strong and airtight to prevent any damage to the sensitive electronics it contains.

A DOM is generally constructed out of glass allowing for the PMTs to work without obstruction and along with the PMTs also contains the associated readout channels. The

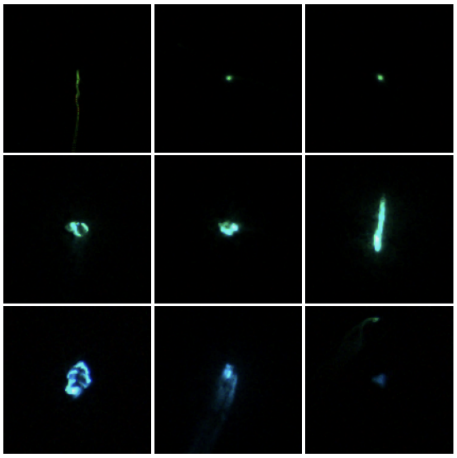
[*]https://www.hamamatsu.com/us/en.html

[†]See Appendix B for a more detailed breakdown

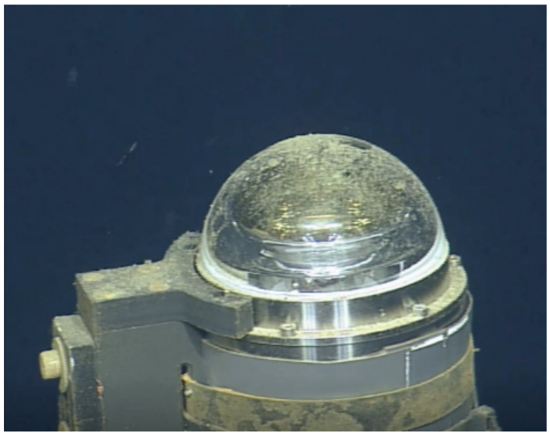
DOM also obviously needs to be constructed in a way that allows for the PMT signals to be sent out of the DOM for further use. [43]

Sea-based detectors face many unique challenges one of which is bio-fouling, this is the natural process of organisms using the surface of the DOMs as their habitat. This would lead to a steady drop in the amount of data we would receive from the DOMs so it would be necessary to have regular maintenance of the DOMs to maintain functionality in the detector. [44, 45]

Not only does the life present in the sea reduce the efficiency of our detectors through bio-fouling but also presents us with a unique problem where certain species of bioluminescent organisms emit light with wavelengths ranging from 450 nm to 500 nm [46–48] while the peak Cherenkov wavelength is expected to be ~420 nm [49] and unlike the electrical noise is significantly harder to filter out since the behaviour of these organisms is far more correlated along with being less well studied. Another source of noise at this level is the presence of Potassium-40 (K-40) however just like the electrical noise this is uncorrelated and is again relatively easy to treat.



(a) Different species of Bioluminescent Organism that will inject noise into the data collection process.



(b) Bio-fouling

Figure 3.4: Illustration of the environmental effects on STRAW a P-ONE pathfinder mission to study the environment of the selected detector location. [44]

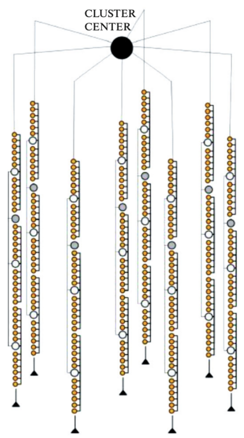
3.3 String

Strings can essentially be described as a chain of DOMs. Here again, there is a difference in how the strings are set up based on which medium the detector is in. When it comes to IceCube the strings are in ice, so a sufficiently deep hole needs to be drilled and then the DOMs may be dropped into place, on the other hand, sea-based detectors need to take advantage of buoyancy to keep the string in place by first anchoring the string to the sea floor and then making sure the rest of the string remain taut by attaching a buoy to the top of the string.

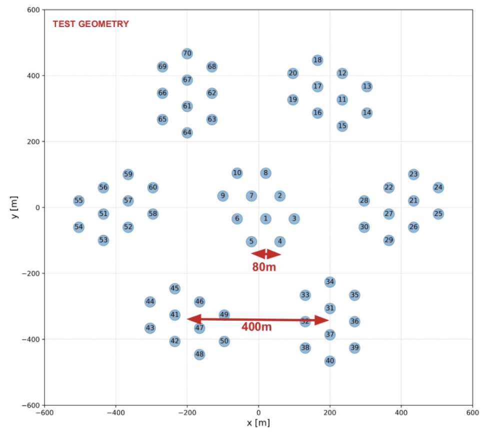
DOMs are generally evenly spaced throughout the string and with some buffer space at the top and bottom. The string is equipped with a data transfer cable to transfer the data from the PMTs to first the junction box and much later to a server on land. It also quite naturally needs to be equipped with a power line which will supply the PMTs with the power required to function. [45]

3.4 Cluster

This is an ”optional” feature for neutrino detectors in the sense that this heavily depends on the medium the detector is in. For example, IceCube does not have a cluster architecture because the optical features of the ice vs the reduced maintenance requirements make it far more convenient for it to be in a grid. Sea-based detectors on the other hand are in a different optical medium and more importantly need to be spaced, allowing underwater robots enough clearance to access all the strings for maintenance. This architecture also allows for deployment of features for example a trigger at an intermediate level in between string and detector level. [50]



(a) Baikal Cluster [51]



(b) Potential P-ONE Cluster [52]

Figure 3.5: Cluster layout

3.5 Detector

This is the most big picture view of the detector and at this level, we need to make sure that certain features are present in the detector somewhere while they don’t need to be present in every string or even in every cluster. Some of these features include an ocean current measurement device, a mechanism for the DOMs to measure their relative positions*, monitoring the optical properties of the ice/water. At this level, we also need to take into account how the entire detector power will be managed along with the fact that we need to make sure that the detector has sufficiently filtered out the signal from the noise and the bandwidth to transfer the signal to a server on land for later analysis. Things such as environmental impact, cost, maintenance and other big-picture features need to be decided on at this level. [45]

*the mechanism of this device may be acoustic or light based

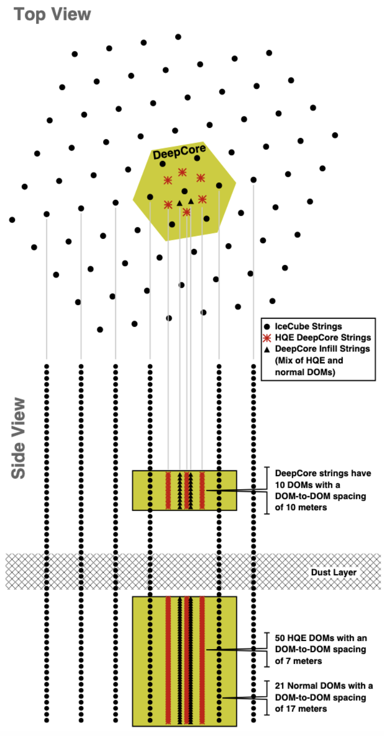


Figure 3.6: IceCube Detector [53]

Design of TRIDENT

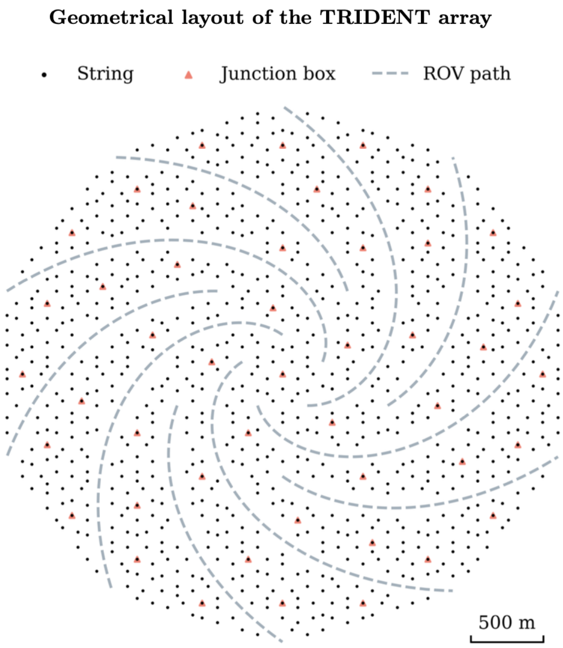


Figure 3.7: Planned TRIDENT Design [50]

So to conclude this part lets summarise what we learnt, a neutrino telescopes works by placing string in a specific configuration based on real world design constrains in massive volumes of water or ice, the string is made up of DOMs that house PMTs which act as the smallest detecting unit of our detector. After this our detector just waits for neutrino events to happen around it as it constantly deals with various sources of noise.

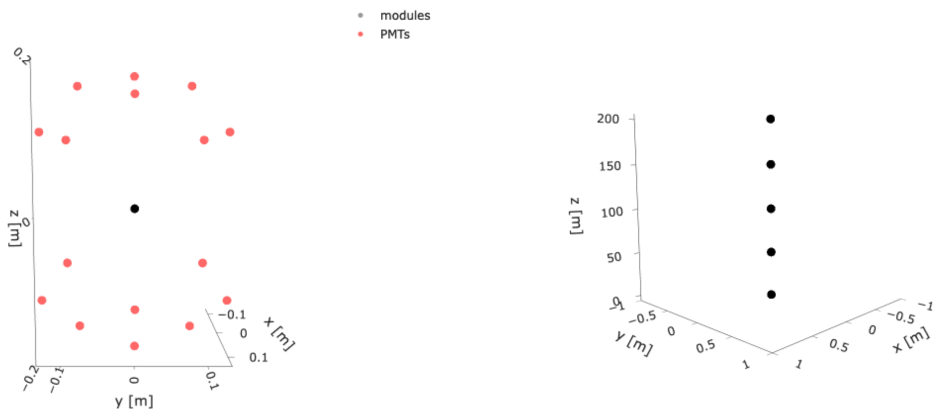
4 Simulation Software

The simulation and data handling are done by Olympus and Ananke, two open-source Python packages that are built on top of various other packages to ensure high performance and efficiency along with providing various useful utilities.

4.1 Ananke

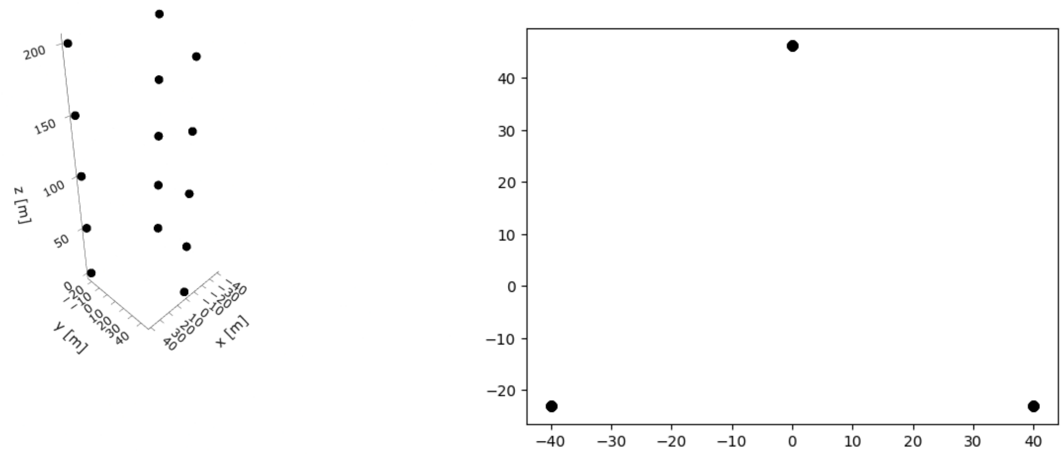
Detector Simulation

Ananke simulates the entire detector response. Going from the smallest part to the largest. The first thing defined is the PMT which is the the part of the detector that does the detecting work. It is defined by 3 characteristics: PMT Area, PMT Noise Rate and PMT Efficiency which define how big the PMT is, how much dark noise it will produce in a simulation and efficiency will define how many photons from the events and bioluminescent noise are incident on the PMT will be saved. One level above we have the DOM, the DOM consists of 16 PMTs evenly spaced out in a spherical configuration as seen in fig Figure 4.1a. The DOM is defined by its radius and then the PMTs are automatically spaced out based on this information. Another level above that we have the cluster geometry where we define how many DOMs we have on the string (see Figure 4.1b) how many strings we want and how they are organised. Some examples of detector presets available in the software are Figure 4.2 - Figure 4.5



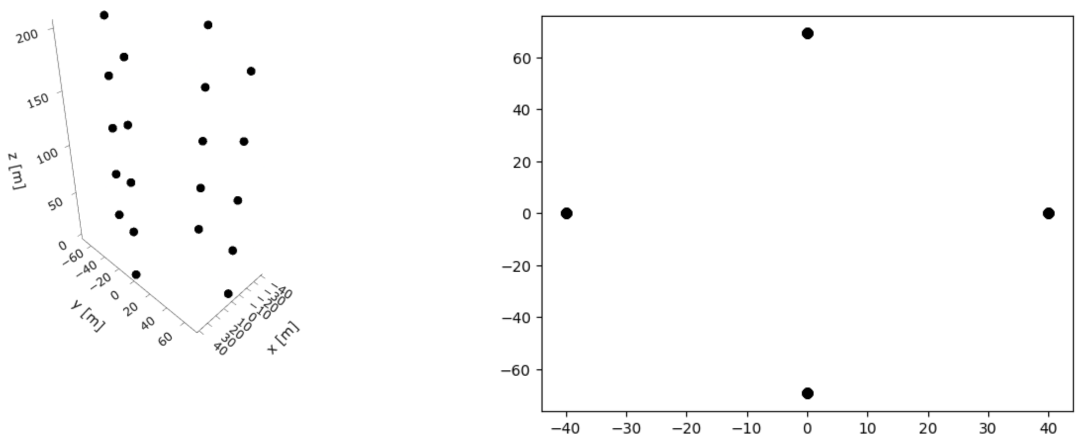
(a) PMT Orientation in a DOM (b) Example of a string with 5 DOMs

Figure 4.1: Visualization of PMT Orientation and a String with 5 DOMs



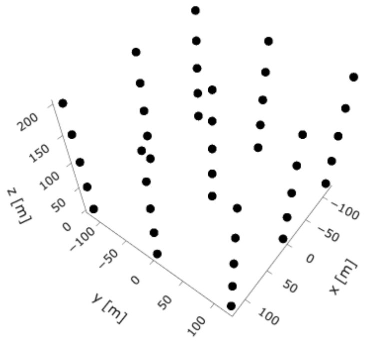
(a) 3-D View (b) Top View

Figure 4.2: Strings placed in a triangular grid

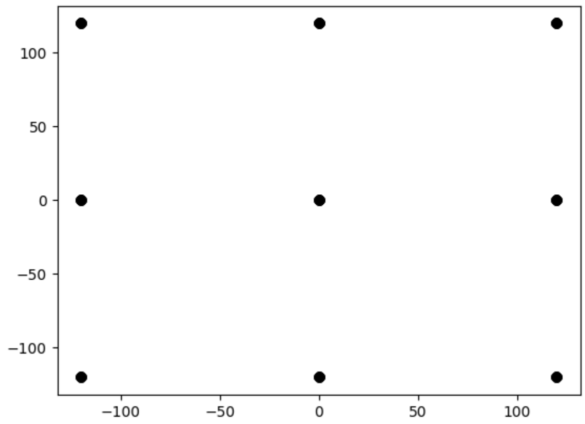


(a) 3-D View (b) Top View

Figure 4.3: Strings placed in a rhombic grid

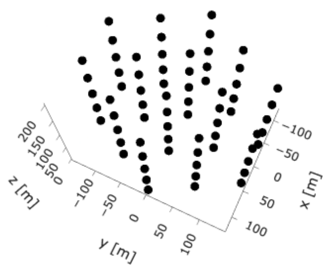


(a) 3-D View

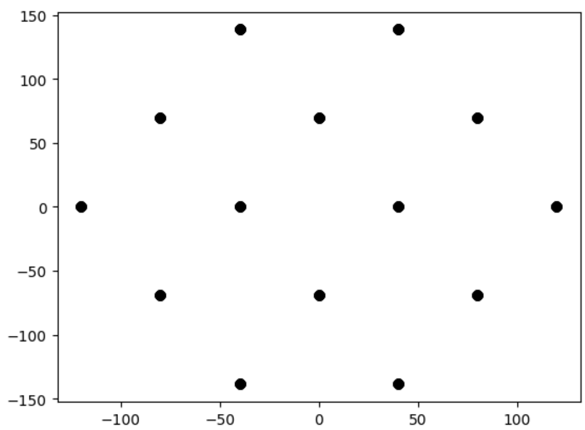


(b) Top View

Figure 4.4: Strings placed in a square grid



(a) 3-D View



(b) Top View

Figure 4.5: Strings placed in a hexagonal grid

Data Management

Ananke is also responsible for creating HDF5 files and saving the simulation data in these files efficiently. Each file contains the detector configuration, event simulation data, detector response and a full event summary saved internally as detector, hits, sources and records respectively. Information about the data saved in each column of each data set is given in Appendix A It also allows one to merge data sets and redistribute the hits. The visualization of this can be seen in Figure 5.3

4.2 Olympus

Olympus is responsible for managing the simulation and hit generation half of the pipeline. It is capable of simulating Neutrino Events, Electrical Noise and sampling Bioluminescent Noise.

Neutrino Events

Neutrino events can be further divided into Cascades, Starting Tracks and Realistic Tracks. All neutrino events start with picking a random point inside a specified volume where a CC interaction occurs within a specific energy range and then the muon is propagated in a random direction and at each time step of the location of the muon is used as a source of Cherenkov photons. These photons are then propagated out in a cone where they may intersect with the PMTs in our detector. The photon and muon propagation is constrained by user-defined properties of the medium in which the event is occurring. Also, it should be noted that the only difference between Starting and Realistic Tracks is that the volume in which the Starting Track* start always start inside the detector volume vs Realistic Tracks may have origins outside the detector as well. Visualisations of these simulations will be seen in Chapter 5 in figure Figure 5.2

Noise Generation

Electrical noise is generated for each PMT, this is done by sampling from a Poisson distribution that's defined by the noise rate of the PMT. Olympus does not simulate its own bioluminescent noise but rather sample noise pre-generated using software called Fourth Day [55]. The version of Fourth Day Olympus simulates different species of organisms that emit a flash upon encountering a DOM. These organisms are sensitive to shear forces, such as those experienced when encountering a DOM, and each species has its own unique distribution from which it samples.

*There are reasons other than saving computational resources as to why we would be interested in these [54]

5 Simulation

5.1 Simulation features

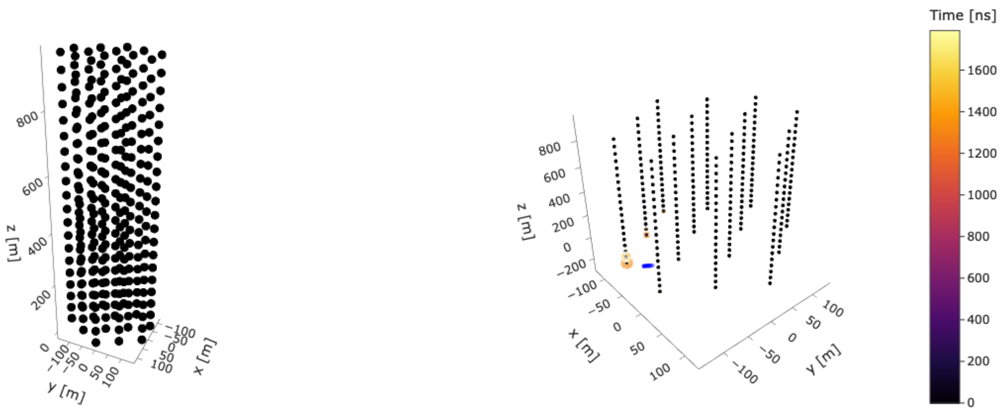
At this point, there is only one thing stopping us from running our simulations and that is the fact that we have not yet finalised the detector configurations for our simulation. While we are primarily concerned with P-ONE for many features its final designs have yet to be finalised. Features such as string spacing and string configurations, the number of DOMs, the length of the string, etc, are very important and will affect our simulation. So to get a good understanding of what the acceptable values for our detector simulation can be we surveyed other neutrino detectors (Appendix B) and finalised our simulation features as can be seen in Table 5.1

Table 5.1: Detector Configuration Parameters

Parameter	**Value**
Modules per line	20
Vertical distance between modules	50.0 m
Dark noise rate	16×10^{-5} ns^{-1}
Module radius	0.21 m
PMT efficiency	0.42
PMT radius	37.5 mm
Geometry type	Hexagonal
Minimal String Spacing	80 m

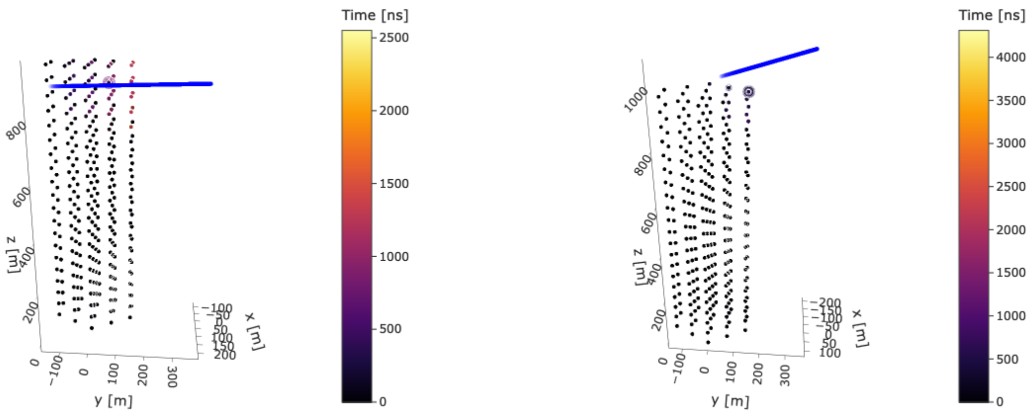
5.2 Simulation Visualisation

In this section, we will go over the results of our simulation. Figure 5.1a is the grid we use based on Table 5.1. Figure 5.1b and Figure 5.2 show us the propagation of our source through the medium and the times at which the DOMs are being hit by the photons emitted by the source.



(a) Grid Visualisation based on Table 5.1 (b) Cascade event

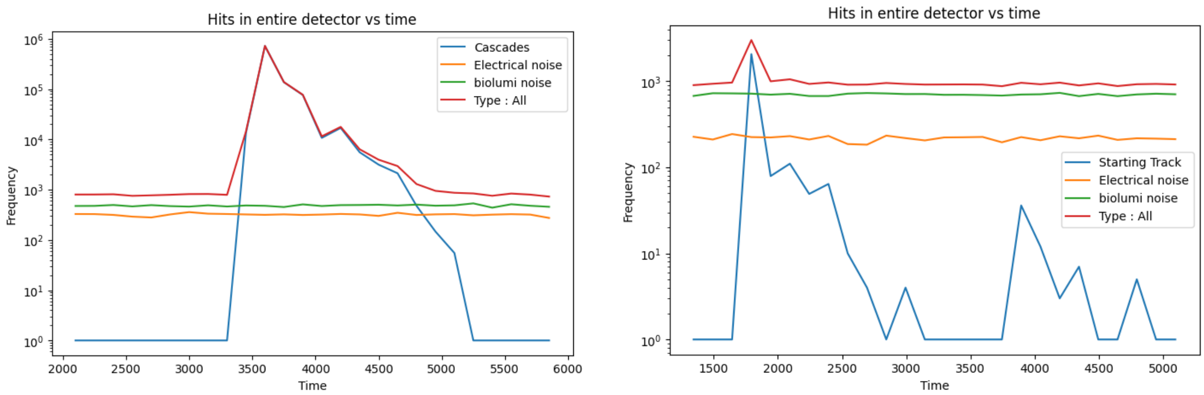
Figure 5.1: Visualization of the detector and a Cascade type simulation



(a) Starting Track Event (b) Realistic Track event

Figure 5.2: Visualisation of simulated events

Next, we see in figure Figure 5.3 what the hit count vs time for a single simulated event that has been injected with 4,000 ns of noise looks like. As we can see for cascades the hits are fairly concentrated in time while tracks are more spread out this is because cascades travel a very short distance and are only capable of interacting with a few detector strings while a track tends to interact with many more so the hits it generates in our detector are allowed to be spread out in space and time. We can also make this observation by looking at Figure 5.2a where the track passes by many strings while in Figure 5.1b the event is confined to a much smaller region of the detector.

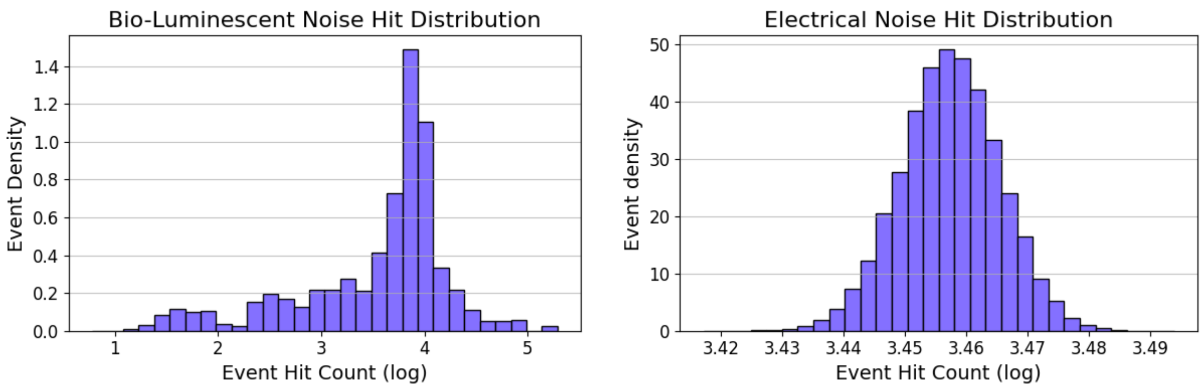


(a) Cascade merged and redistributed with the noise

(b) Starting Track merged and redistributed with the noise (time in ns)

Figure 5.3: Visualization of detector response for 1 event

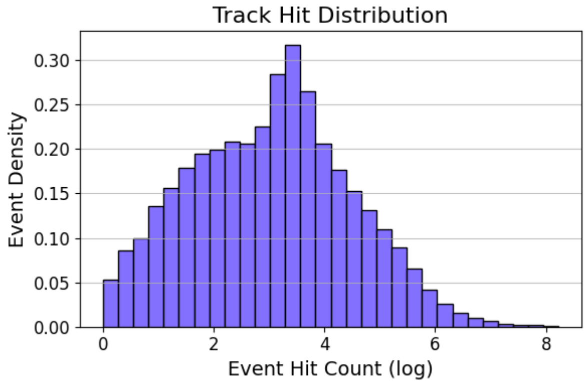
The distributions we will see from now on will involve 140,000 track-type simulations and 300,000 cascade-like simulations. We can see the distribution of hits for our simulated noise and events in Figure 5.4 Figure 5.5 respectively, while in Figure 5.6 and Figure 5.7 we can see how the distribution changes as 4,000 ns and 100,000 ns of noise are injected into the simulated events respectively. And as we can see as the amount of noise injected into the simulation increases our distribution moves right. As a sanity check, we can see the amount that the distribution shifts to the right Figure 5.6 corresponds with the distributions in Figure 5.4.



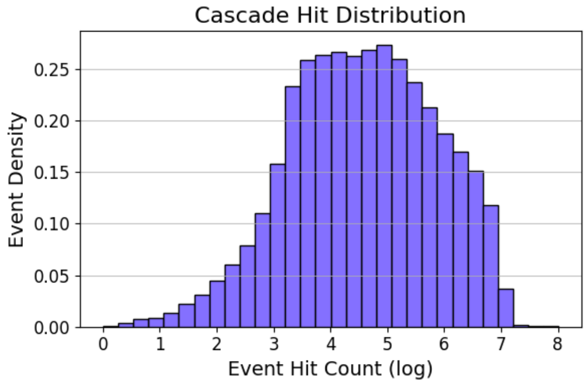
(a) Bioluminescent Noise Hit Distribution (4,000 ns)

(b) Electrical Noise Hit Distribution (4,000 ns)

Figure 5.4: Noise Hit Distribution

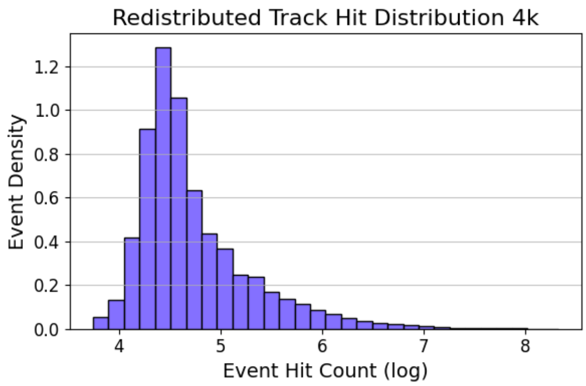


(a) Track Hit Distribution

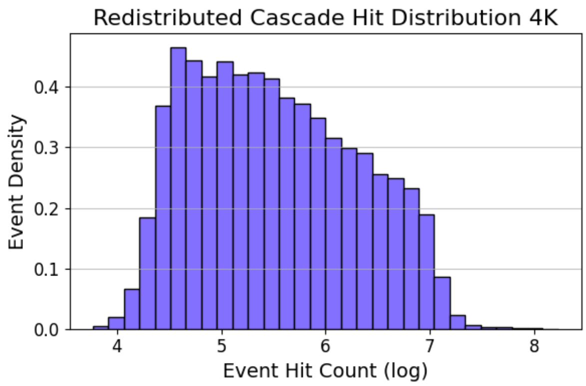


(b) Cascade Hit distribution

Figure 5.5: Pure Event Hit Distribution

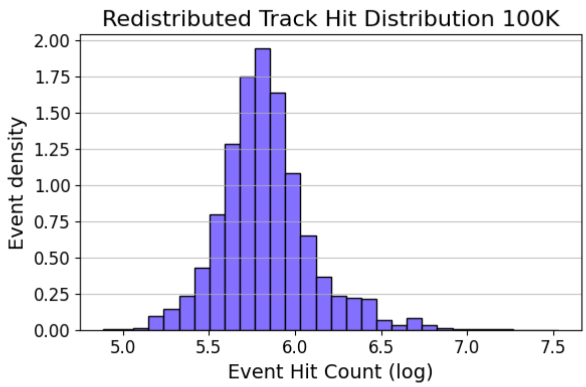


(a) Tracks events injected with 4,000 ns of noise

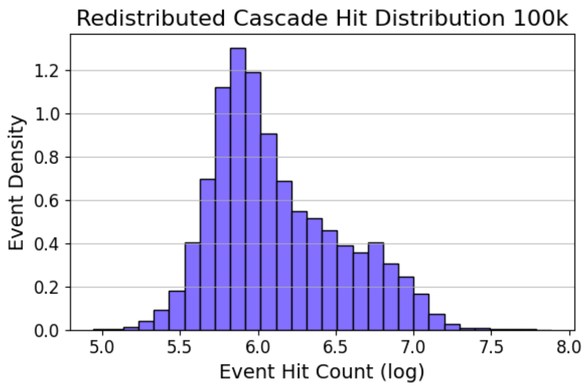


(b) Cascade events injected with 4,000 ns of noise

Figure 5.6: 4,000 ns of noise



(a) Tracks events injected with 100,000 ns of noise



(b) Cascade events injected with 100,000 ns of noise

Figure 5.7: 100,000 ns of noise

Now we can take a look at the Hits vs the Energy, Figure 5.8 and Figure 5.11 help us see this distinction between Cascades and Tracks as Cascade-type events have a much stronger correlation with energy compared to Tracks. We can also see that this correlation gets destroyed as more noise is injected into the simulation. The already weak correlation for tracks is almost immediately destroyed and we essentially see the distributions from Figure 5.4 again just smeared out over the entire energy range. While for cascades 4,000 ns of noise seems to overwhelm the relation at the low energy hits end of Figure 5.12 on the higher end we can still quite easily see the trend. Even with 100,000 ns of noise the trend is not completely destroyed in Figure 5.13.

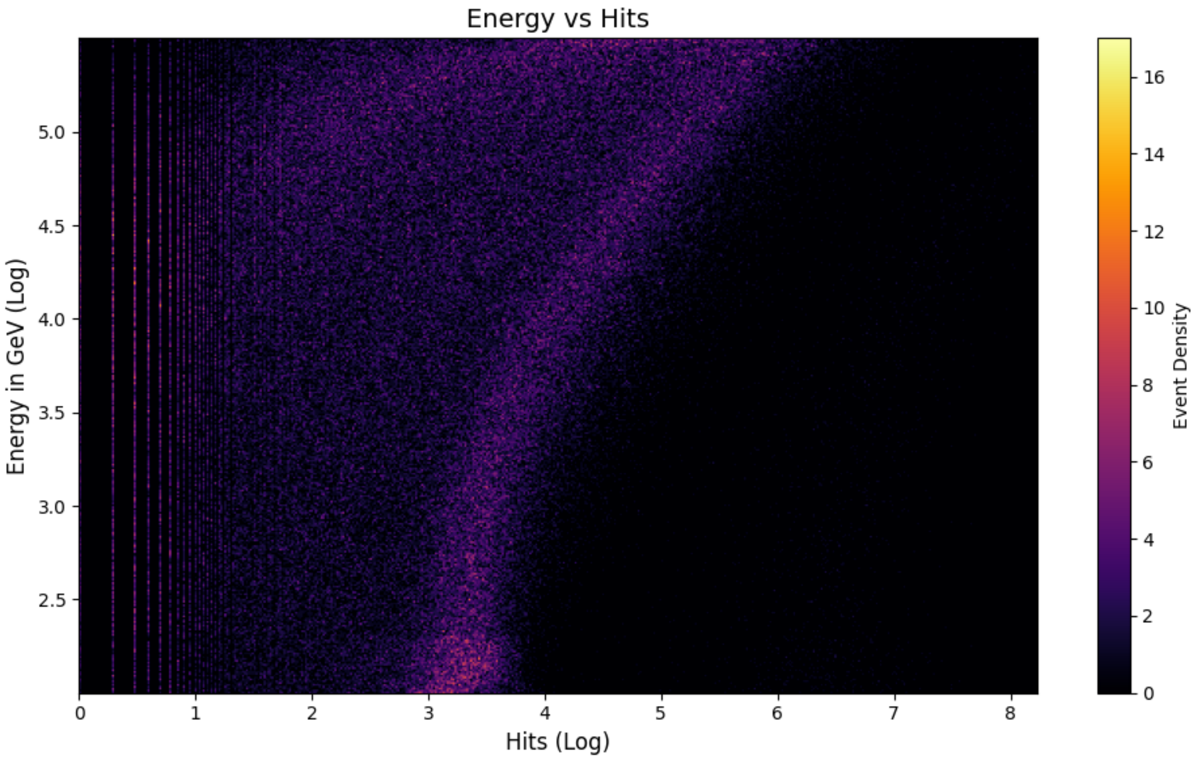


Figure 5.8: Hits vs Energy for Track-type events

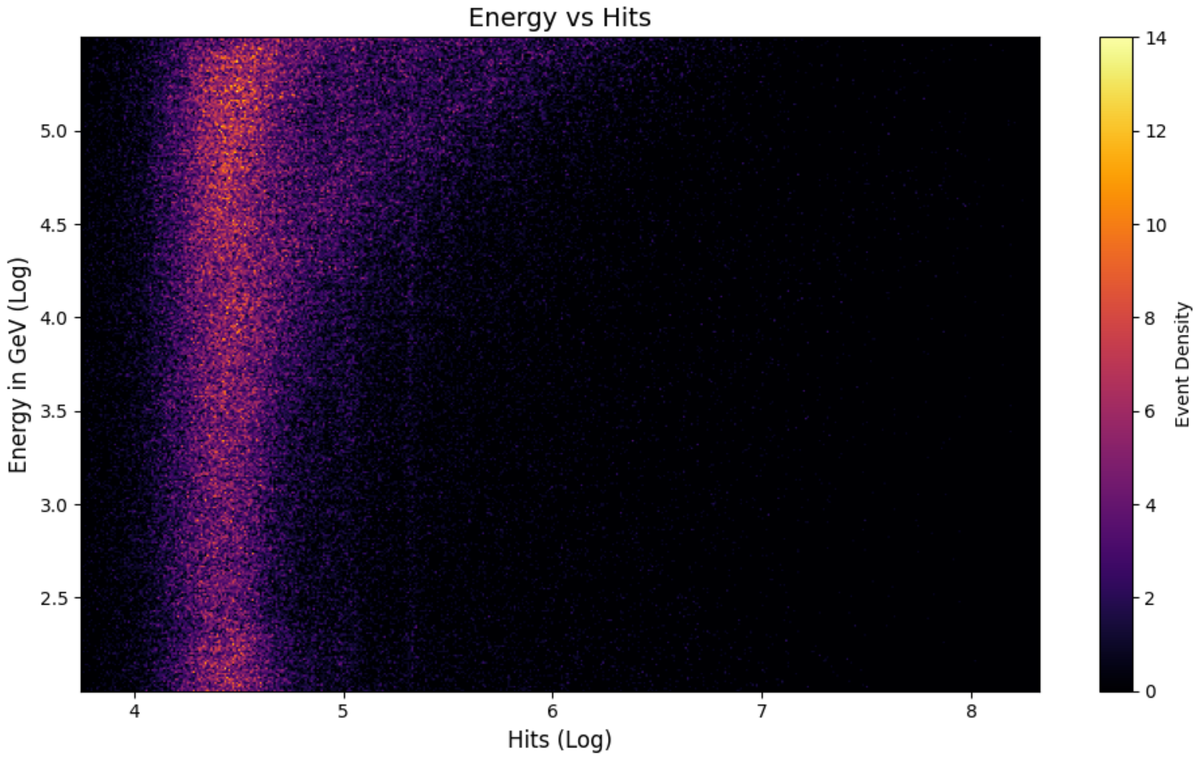


Figure 5.9: Noisy hits (4,000 ns) vs Energy for Track-type events

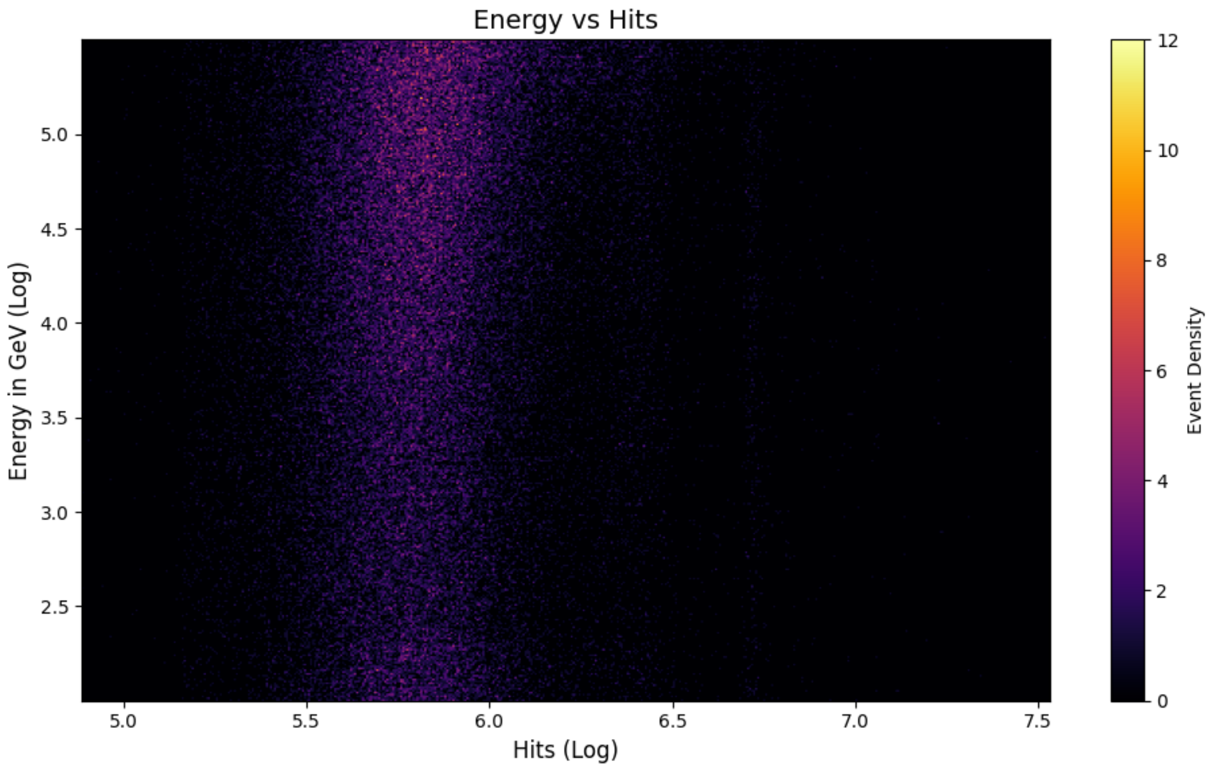


Figure 5.10: Noisy hits (100,000 ns) vs Energy for Track-type events

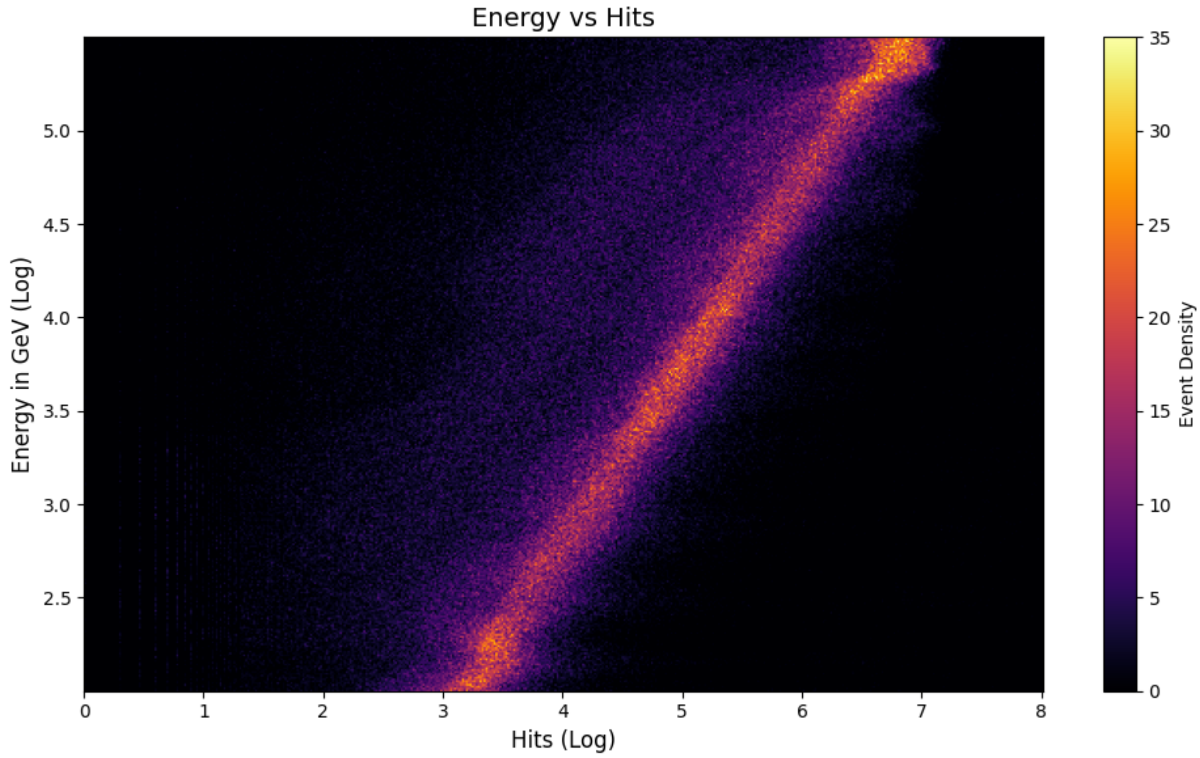


Figure 5.11: Hits vs Energy for Cascade-type events

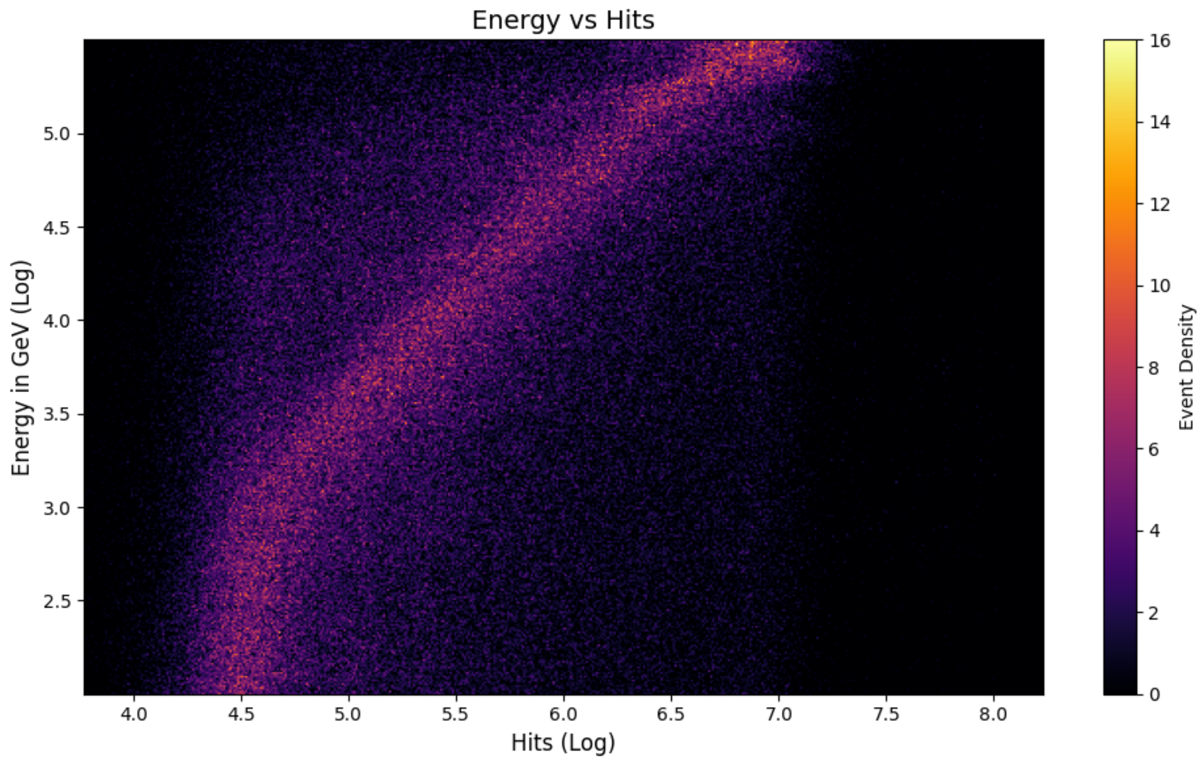


Figure 5.12: Noisy hits (4,000 ns) vs Energy for Cascade-type events

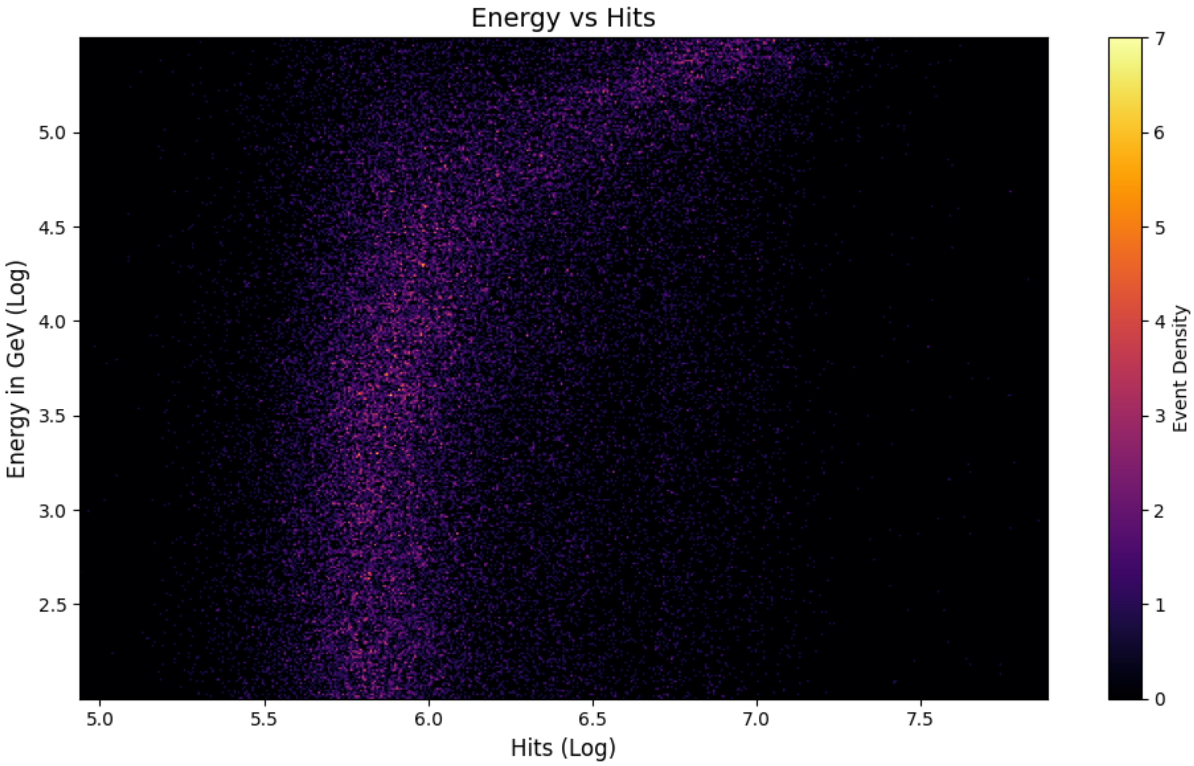


Figure 5.13: Noisy hits (100,000 ns) vs Energy for Cascade-type events

6 Coincidence trigger

6.1 The Algorithm

The Coincidence trigger or the L0 trigger is the simplest possible trigger we can develop and will serve as a baseline to see how well or poorly the GNN functions. To begin with, the coincidence trigger works at the DOM level and hence is extremely fast. The idea is fairly simple, we expect an event to light up more PMTs inside the DOM than noise would. To make this intuition a little more solid, let's get some quantities to help us here. The entire algorithm has 2 parameters that define its behaviour, the time interval and the coincidence level.

The coincidence level(CL) is the number of PMTs that need to be hit within a given time interval for the DOM to decide to keep the data. So for our DOM with 16 PMTs, it is obvious that the optimal CL is somewhere in the middle but the exact value that should be picked is not obvious to us from the start. So in (Section 6.3) we experimentally verify what is the optimal value for our algorithm.

The time interval on the other hand is just as the name suggests the interval of time in which the DOM will collect data and then decide if it's going to keep the data or throw it away. The time interval can't be too long since that would increase the likelihood of when the noise accidentally creates hits in enough PMTs to exceed the CL threshold. But the time interval should also be long enough that all the PMTs that could get hit by a neutrino event do get hit. So based on the fact that light takes 10 ns to go from one end of the DOM to the other, this value was chosen as our time interval. With this our trigger can now be used on our data. The next few sections will be spent trying to understand how our trigger behaves.

6.2 Trigger Response

In Figure 6.1 we can see the response of our trigger to a simulated event. The event has been injected with 6000 ns of noise resulting in us having 600 intervals of 10 ns each. The rightmost graph shows the simulated signal while the middle plot shows the hits from the DOMs that were triggered, just by observation we can see that the trigger works as expected. Notice that in the rightmost graph the baseline number of hits $\sim 10^2$ which from our discussion in Chapter 5 is the expected amount of noise in the signal and once the trigger has been applied the baseline goes to 0 while the peak of the signal remains unchanged. However, this is a fairly strong signal for events with fewer hits significant portions of the signal also get lost.

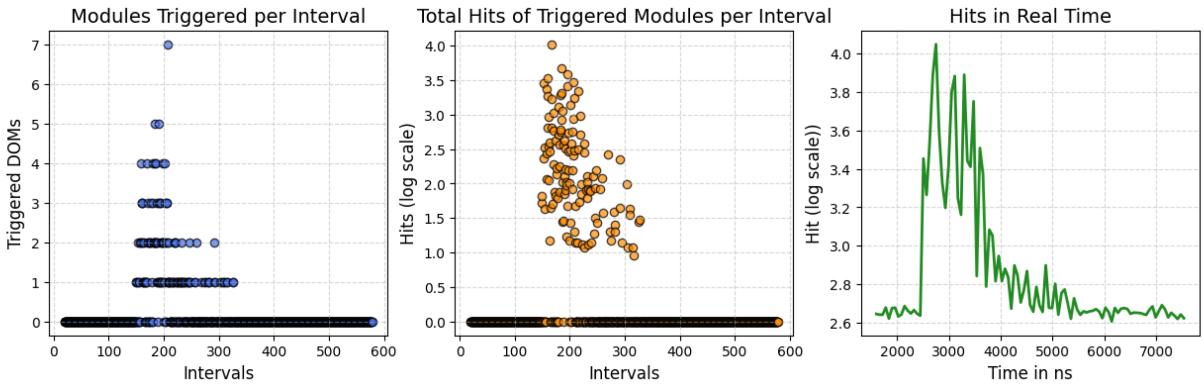
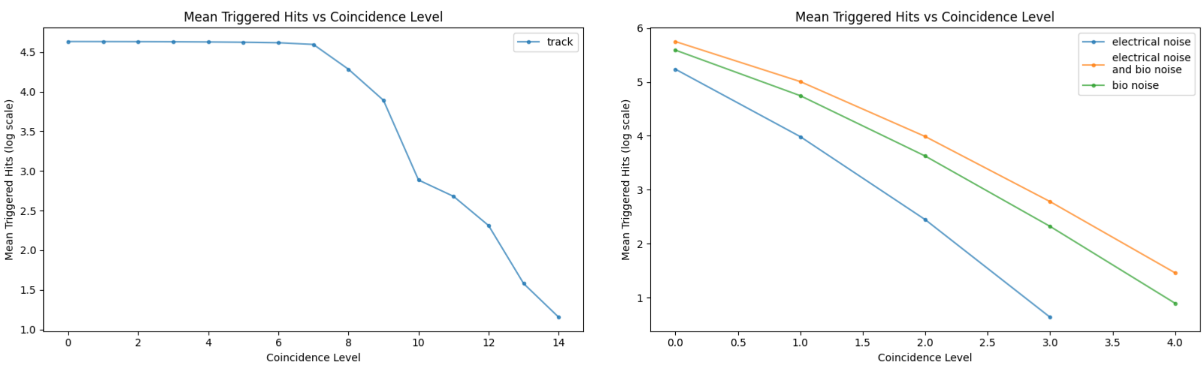


Figure 6.1: Trigger Response Example CL=7

6.3 Coincidence trigger study

Now to understand what the best coincidence level is for our purposes we will apply this trigger to our Track dataset. Note that similar arguments can be made for cascades however the case for tracks is more illustrative due to their relatively low hit counts. In Figure 6.2a we can see that the hit counts for Tracks essentially remain unchanged up until CL=8 where the number of hits begins to drop dramatically. This is to be expected since on average an event should light up at most 1/2 the DOM while rarely we will have an event that might pass through the DOM or some other edge-case scenario. On the other hand, we see that the hit counts in our simulated noise* in Figure 6.2b starts to fall immediately and by CL 4 is negligible with the rare possibility of the noise surviving past this point.



(a) Track

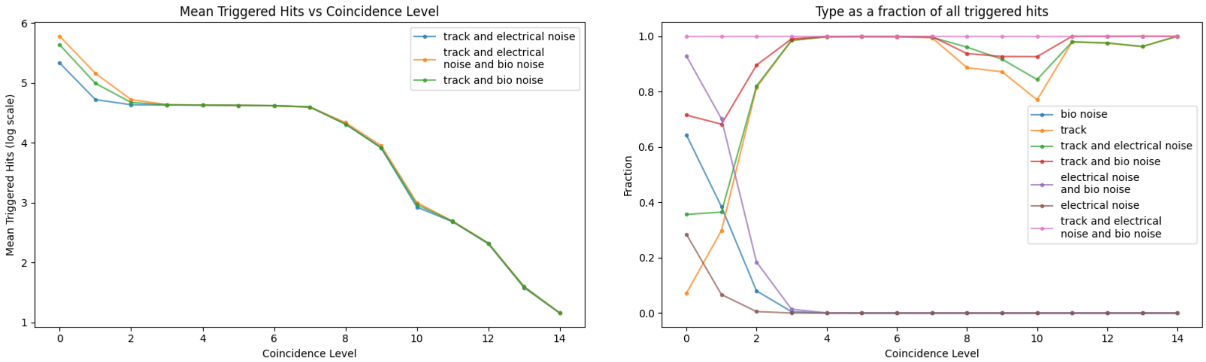
(b) Noise

Figure 6.2: Mean response of trigger to simulated events

So now we have an intuition that the optimal region is somewhere between 4-7 inclusive. We can then check this by seeing Figure 6.3a where we see that for noisy events

*Note that the absolute number of hits here is less instructive than the shape of the graph since the number of hits here is dictated by the amount of noise we put into the simulation. However, since there is a maximum amount of noise we expect to see until another neutrino event occurs[56] in the detector we don't need to worry about this.

from CL=0-4, the graph looks similar to the pure noise graph while after that point the graph looks identical to that of the pure track. However, it's still unclear if there is an optimal choice. To understand this we look at Figure 6.3b where we see what fraction of the triggered hits can be associated with a specific source as we would expect for CL=0-4 the majority of the triggered hits are noise but as we increase the CL the track hits start to dominate, however interestingly past CL=7 we see that noise again start playing a part as tracks that might only have hit a few PMTs along with the noise can cross the threshold indicating that past CL=7 not only do we unnecessarily loose event information but that the is also much noisier than the in our optimal range. This however does not indicate if any of the acceptable CL are better than each other. Since the CLs are equally good for the rest of this thesis we will carry on using CL=7 knowing that any other CL would give us an equally good result*.



(a) Tracks combined with noise

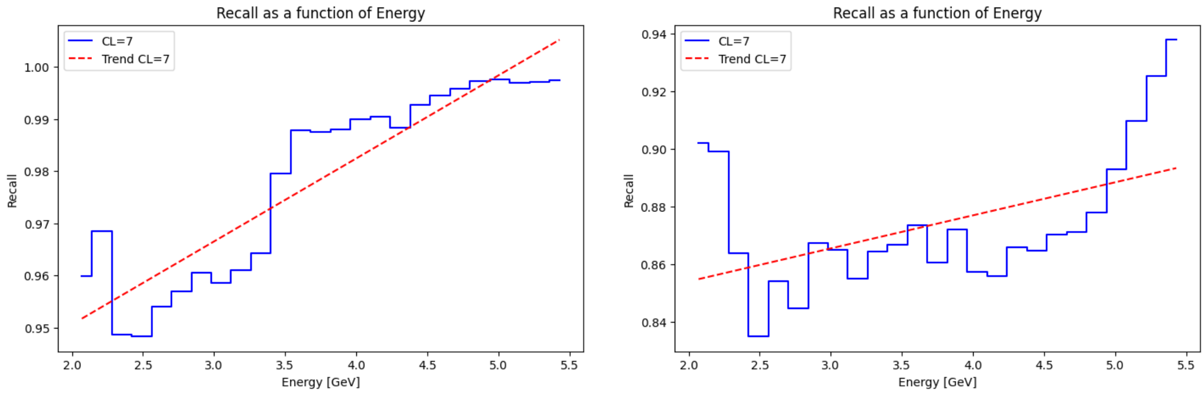
(b) Type of hits as a fraction of all triggered hits

Figure 6.3: Trigger Response to Noisy Events

6.4 Event Classification Ability

Finally, we want to investigate the energy dependence of the trigger so in Figure 6.4 will study what fraction of all the simulated events were triggered also known as Recall† and its relationship to energy. As we can see the trigger is better at detecting higher energy events and better at detecting Cascades than Tracks. This is to be expected based on our discussion in Chapter 5 since the number of hits for a track is less strongly correlated with energy and also has fewer hits in general, a fact that our coincidence trigger would naturally be sensitive too. We also see fluctuations in the Recall value with energy, in particular, the Recall value at the very beginning is higher than the following energy bins. Since the plot is showing us a ratio the underlying energy and hit distribution should not affect the plot, so it is unclear if this is because of something systematic or random.

*These findings also consistent with KM3Net Experimental findings.[43]
†Recall will be discussed in some more depth in Chapter 8



(a) Cascade (b) Track

Figure 6.4: Fraction of Triggered Events as Function of Energy

7 Introduction to Graph Neural Networks

7.1 An Abridged Introduction to Machine Learning

"A computer program is said to learn from experience E concerning some task T and performance measure P if it's performance on T improves with experience E." [57] As the quote neatly lays out machine learning is about teaching a model to take inputs then making it do a task based on the inputs to get an output and then finally evaluating how well the model did the task based on this output and finally giving the model feedback based on this evaluation changing the model in this process i.e. learning and finally repeating this process all over again for a specified number of iterations/epochs or till the model has learnt to perform the task to a satisfactory level.

Just as there are different ways in which humans learn similarly there are different ways in which our model can learn, the method of interest for us is Supervised Learning. The philosophy behind supervised learning is like a human trying to learn something exclusively by taking tests. There is a test and an answer key or in the more standard jargon features and truth and once the model has taken the test it is given a grade and told where it went wrong. This is in contrast to unsupervised learning where an answer key doesn't exist and the model learns based on trying to learn from patterns hidden within the data, but these are a completely different class of methods and are not relevant to us.

Coming back to supervised learning there are generally 2 tasks that can be performed using this method, Regression and Classification. The task of interest to us is classification since the entire point of this endeavour is to classify whether some of the hits seen by our detector come from neutrino events or are noise. And at this point we can talk about how we are going to evaluate our model. Like with all tests there are a variety of grading schemes to choose from similarly we will evaluate our model outputs using something called a Loss Function.

Loss Function

Loss functions have different names in different fields including Risk, Reward, objective function, etc. The point of this mathematical object is to take the predictions made by the model and the truth and then give us a number that tells us how close or far the model's predictions are from the desired output. The model's singular purpose is to minimize this loss function. There is extensive literature on the various loss functions and what they are good for but for our purposes, we will only need one and that is the Binary Cross Entropy Loss Function.

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\right] \tag{7.1}$$

To understand why this is the tool for this task we need to understand what we are doing with our data. To be able to classify our data we are going to label neutrino events as 0 and noise as 1 so when these are our labels which in the Equation 7.1 is denoted by y_i while $\hat{y}_i$ denotes the prediction our model has made. So based on what the label is either the 1st or the 2nd term will be 0. For concreteness let's say $y_i = 0$ then we only have the second term. Now we can evaluate our prediction, it can be seen that as $\hat{y}_i \to 0$, $\log(1 - \hat{y}_i) \to 0$ and as $\hat{y}_i \to 1$, $\log(1 - \hat{y}_i) \to -\infty$ this is precisely the behaviour we a looking for from out loss function since it allows from the differentiation between our label and blows up when the model makes a mistake. In practice when using loss functions it doesn't make sense to allow $\mathcal{L} \to \infty$ so the loss function is usually capped at a sufficiently large value to indicate the badness of the prediction.

Typical Approach to Machine Learning

So any ML problem can be divided into the following parts: Problem Definition, Data Collection, Data Preprocessing, Model Selection, Model Training & Tuning and finally Evaluation. So far we know what our problem is, we spent all of Chapter 5 discussing data collection, and we jumped ahead a little with our discussion of loss functions which is a major part of training and tuning but not quite everything. The missing pieces are Forward Pass and Backpropagation. So now we must understand what's at the heart of modern ML and this will then inform us on our decisions when it comes to the topics that have not been covered so far.

Neural Networks

Neural networks are made up of neurons (or nodes) and consist of three types that are associated with the different layers of the neural network. The three types are the input layer, output layer, and hidden layer.

The input layer is responsible for taking in the data and passing it on to the hidden layer. The input layer consists of the same number of nodes as the number of inputs given to the network. For example, if the input is a matrix with 32×32 dimensions, the input layer would have 1024 input nodes in the input layer. Mathematically, we can think of the input layer as a single input vector $\mathbf{x}$ with $(n, 1)$ dimensions, where n is the number of inputs.

The action of a single hidden layer can be broken down into two parts. The first part is the application of weights and biases:

$$\mathbf{z_i} = \mathbf{W_i x_i} + \mathbf{b_i} \tag{7.2}$$

Where i is the layer number, $\mathbf{W_i}$ is a matrix with dimensions (m, n) where m is the number of nodes in the hidden layer, and $\mathbf{b_i}$ is a vector of dimensions $(m, 1)$. The values for the weights and biases are randomly set at the start of training and, in theory, can be any real number. However, in practice, they are usually constrained to small values for ease of computation. Weights tell us how important certain features/combinations of features are while the bias term can be thought of as a way to control how strongly the the weights will affect the next layer.

The second step introduces non-linearity. So far, our calculations have been linear, which would be sufficient if all our data could be separated linearly but usually, that's not the case. Using exclusively linear functions would effectively be like training a model with a single layer. Adding non-linearity allows us to stack hidden layers, with each layer learning more complex features. The activation function σ_i is applied element-wise to $\mathbf{z_i}$:

$$\mathbf{x_{i+1}} = \sigma_i(\mathbf{z_i}) \tag{7.3}$$

σ_i can be any non-linear function including z^2 or $sin(z)$ but again for practical reasons, we use more functions like ReLU, Sigmoid, tanh etc [58]. By applying an activation function, the network can learn non-linear relationships, making it much more powerful than a purely linear system. Since σ is applied element-wise it is allowed to be different for different nodes.

Finally, the output layer is just the final hidden layer and consists of as many nodes as the number of features we are interested in, for a binary classification task you may go with 2 output nodes with each node associated with a possible output or a single node where we can use the Sigmoid.

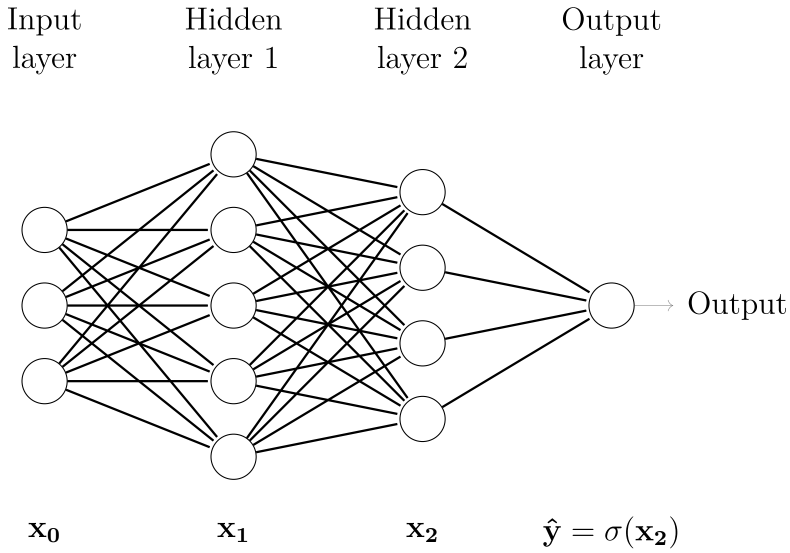


Figure 7.1: Example of a NN for Binary Classification where σ in this case is the sigmoid function giving us probabilities

The entire process of going from $\mathbf{x_0}$ to $\sigma(x_2)$ in Figure 7.1 is called forward propagation at which point we feed $\sigma(x_2)$ to our loss function and we evaluate how good or bad the prediction was now we need to find a way to update the model based on this evaluation. That's where backpropagation comes in, we take the gradient of the Loss w.r.t the prediction then we go one layer back and find the gradient w.r.t to the parameters in that layer and we continue this layer by layer, moving backwards through the network until we have computed the gradient of the loss concerning all the parameters. Now we use an updating rule to use the gradients that we just calculated to change the weights and biases and then we are ready for the next round of forward propagation. Updating rules generally involves scaling the gradient by some amount and then adding it to the weights this scaling factor is called the learning rate and what I have been referring to as "updating rules" is generally called an optimization algorithm.

There are various optimization algorithms, but one of the most widely used (including in this thesis) is Adam (Adaptive Moment Estimation). Adam dynamically adjusts the learning rate for each parameter based on past gradients. It makes larger updates when far from the optimal solution and gradually reduces the step size as it converges, leading to a more stable and efficient learning process. [59, 60]. So to reiterate the general procedure is ►Pick the architecture of the Hidden layers of your model. ►Initialise it with random Weights and Biases ►Forward pass ►Calculate Losses ►Backpropagate ►Update Weights and Biases ►Rinse and Repeat

7.2 Global Pooling

Models can easily become overly complex and have too many features and excessive complexity which can lead to problems like overfitting so a way to circumvent this issue is to use global pooling. Global pooling sits in between the last training layer and the output layer where it takes the outputs from the training layer and pools it using some procedure for example taking the average, max, sum, etc. There are as many ways to implement a global pooling layer and can get fairly complex in it's own right. This process drastically brings down the dimensionality of the features learned by the neural network and prevents it from learning features that do not generalise well. [61]

7.3 Recurrent Neural Network

Recurrent Neural Networks (RNNs) are a special type are a special type of neural network that works well for sequential data, so these are particularly relevant to us since we are dealing with time series data. The basic building block of RNNs is a Recurrent unit what makes these nodes special is that they have a "memory" in the sense that prior inputs influence the current input and output. Memory or as it is called more formally the hidden state is calculated using Equation 7.4 where we can see that This hidden state is updated recursively based on the current input and the previous hidden state. The hidden state is then used either directly or, through a further transformation, to predict the next output. Which then can be back propagated to start the learning process. [62]

$$h_t = f(Wx_t + Uh_{t-1} + b) \tag{7.4}$$

7.4 Graph Neural Networks

What are Graphs?

Graphs are a collection of nodes and edges where each node contains information about itself while the edges tell us about the relationship of the node with it's neighbours. To connect it to our detector we can model each PMT, DOM or even each cluster as a node however impractical that might be. So an example of something we could do is let each node be a PMT and then let it contain information about it's hits and let the edges contain the information about it's relationship with it's neighbouring nodes. We could imagine it as the closer the nodes are spatially the stronger their relationships should be.

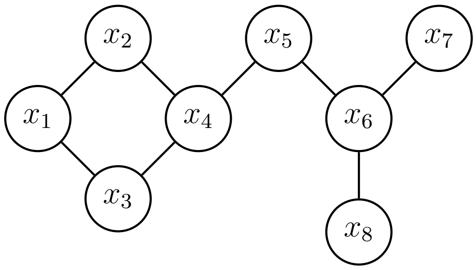


Figure 7.2: An Example Graph

Message Passing

Message Passing is in a sense a forward pass and Backpropagation combined into one thing. When training, a node takes the information(analogous to Forwardpass) from it's neighbours and aggregates this information, computes losses and based on this it updates it's internal state(analogous to Backpropagation). Each of these steps is very model-dependent as there are many approaches to achieve the same results with their pros and cons. So after the first update, all the nodes have some information from their neighbours when the next update occurs through their neighbours they gain the information about their neighbours' neighbours and so on. There is a danger of all nodes finally having identical internal states if we train the model for too long. [63–65]

8 Graph Neural Network Trigger

8.1 Graphnet

Graphnet is a Python based framework that is built on top of Pytorch Geometric that allows for easy implementation of ML Models for neutrino experiments. The goal of this tool is abstract away many of the technical details behind setting up a Model allowing for focus on big picture features and providing modular functionality, this allows for easy combining of various inbuilt features to quickly build models and train them.

To use Graphnet we need to provide it with a detector configuration file, a Data Extraction Module and a Data Conversion Module if your files aren't already converted into Graphnet compatible file formats, namely, Parquet and Sqlite. After you have done these three thing we are set to use Graphnet for training.

To train a model using Graphnet we use the StandardModel function that takes the input of data, architecture and task. While data and tasks are fairly self-explanatory architecture is a little more confusing to understand. Architecture is one of the key features that Graphnet tries to abstract away since it is traditionally one of the more complicated aspects of ML and tends to have a whole host of features that can be tweaked and tuned. Selecting an architecture from scratch is extremely time-consuming since it usually involves trial and error. Hence, Graphnet provides various natively implemented architectures that have been optimised for neutrino telescopes like the ones we have discussed earlier. These architectures take advantage of the detectors to generate the best possible results given the data.

With this brief introduction to Graphnet done with we are ready to see how this tool can be used to develop trigger.

8.2 Data Preprocessing

As mentioned earlier to get started with we need to provide our detector configuration and data extraction module. Along with this since our simulated data natively saves results as HDF5 files we chose to convert it to Parquet to make it compatible with Graphnet. It was also necessary to modify how to data was organised for the sake of training. Graphnet expects that index of the of our dataset should be called event_no. Graphnet uses event_no as the smallest chunk of data it will feed to the model. Our naïve idea was to assign each record_id (Table A.4) as an event_no, this idea was quickly scrapped since we have far too much noise in our dataset for it to be feasible for an entire record to be used in such a manner, it would also not make much sense to do this since ideally, we would want the model to identify which hits originate from neutrino events vs hits due to noise. So instead we decided to assign event_no in a way that would reflect how the model would see the data if it were to be deployed in a detector.

This trigger would have to function at a cluster level this means that firstly the data would need to come from all the DOMs to a junction box and then our model would see the hits from the entire cluster but the model would be constrained by practical considerations such as bandwidth and memory buffers so it would only have a specific amount of time to make this decision. We will need to give this trigger a time interval longer than the 10 ns we gave the Coincidence Trigger (Chapter 6), and it is expected that this time interval should be around ∼100 ns so we decided to just go with a flat 100 ns for this study. So with that, we decided to assign a new unique event_no to hits every 100 ns.

Another data filtration step that we took was to filter out events with too many hits since these events were already very obvious and did not need a fancy trigger to recognise them as an event we decided to filter out all the event_nos that had hits 5σ above the noise level, this would let us focus on events with with low hits or get a more complete picture of bright events by being able to recognise their tails. And the final step is to drop out the excess noise so that, for the purposes of training the ratio of signal to noise is balanced.

8.3 Model Creation

The model we selected for the study is called RNN-Tito which combines Node-RNN and DynEdgeTito and is ideal for events with high levels of DOM activation this is ideal for us since this exactly describes our noisy data. Here we choose to model each node as a DOM containing time-series data and the RNN layer learns the time dependence of our data then we use k-NN (K-Nearest Neighbours), k-NN is generally used to group objects based on certain characteristics [66] however in this case due to RNN output not being necessarily differentiable the grouping is not done dynamically but only calculated once before being passed on to DynEdgeTITO where the model can learn the spatial dependence of our data, finally global pooling is used to summarise the entire Graph before going to the output layer to generate losses and start backpropagation and begin the learning process.

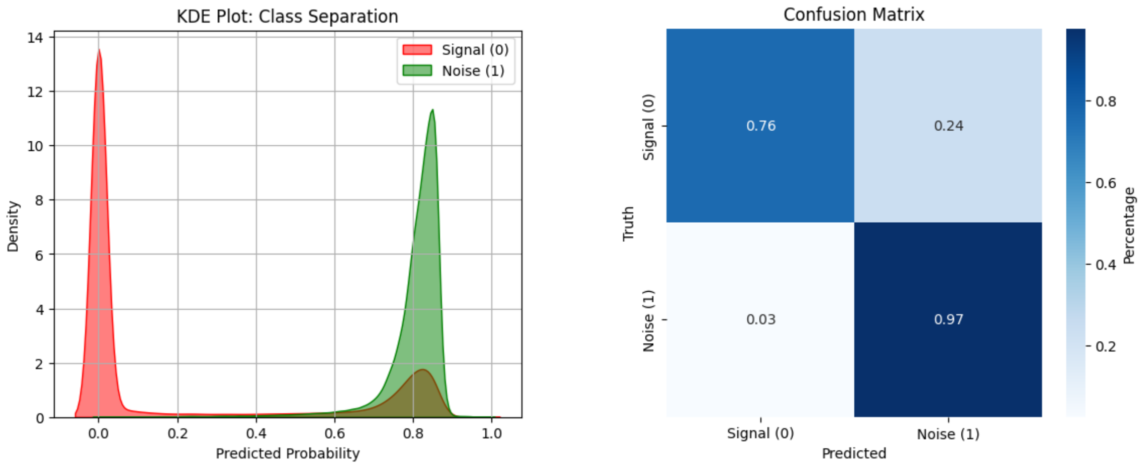
8.4 Results

The following results are from a model trained on a mix of track and cascade data. The first thing to see if the model we have created is doing what you want it to do is to see how the losses change over the epochs. In Figure 8.1 training loss refers to losses related to the data that the model uses to train with which is 90% of the provided dataset while the rest is used for validation i.e. losses on a data set that has never been used for training to prevent model overfitting or underfitting. Overfitting is the process of the model learning overly specific details in the data set that let it perform well in training while performing poorly on the validation dataset because that feature does not generalise usually occurs when a model is trained for too long while underfitting occurs when the model is not trained long enough. Losses that wildly fluctuate also indicate that there might be some issues with the dataset or the model and that training the model is unreliable. For a well-trained model, we expect the Losses to be high in the beginning to fall and stabilise at some point which we see here.

Figure 8.1: Model Losses

To understand how well our model is performing once the training is done we need to get some performance metrics luckily for us there are standard evaluation metrics [67, 68] we can use for this purpose. To start with let's talk about Figure 8.2a, here we can see the distribution of the predicted probabilities the model has given to the events we have given it. The distance from the centre indicates the confidence the model has in its predictions so clearly the events the model does classify as signal/neutrino events it is very confident that they are signals on the other hand it is much less confident in what it considers noise since a significant amount of events we consider signal has also been classified as noise. The strong overlap of signal it has classified as noise with noise also indicates that just training the model for more epochs will not allow for this issue to be resolved since the model genuinely can't find features in the model that would be able to tell the events apart. This behaviour could however be associated with the cuts we have made and a better choice in cuts could provide better results.

The final predictions for the model were made on redistributed simulations that were neither used for training nor validation so the model is truly blind to this data. So on to Figure 8.2b we see the confusion matrix which describes the true positives (TP): a signal that is classified as a signal, true negative (TN): noise that is classified as noise, false positive (FP): noise classified as events and false negative (FN): signal that's classified as noise. Here we take a threshold value of 0.5 and classify all probabilities less than the threshold as events and all probabilities above the threshold as noise and this puts concert numbers to what we saw in Figure 8.2a where nearly 25% of the signal is classified as noise while negligible amount of noise is classified as an event.

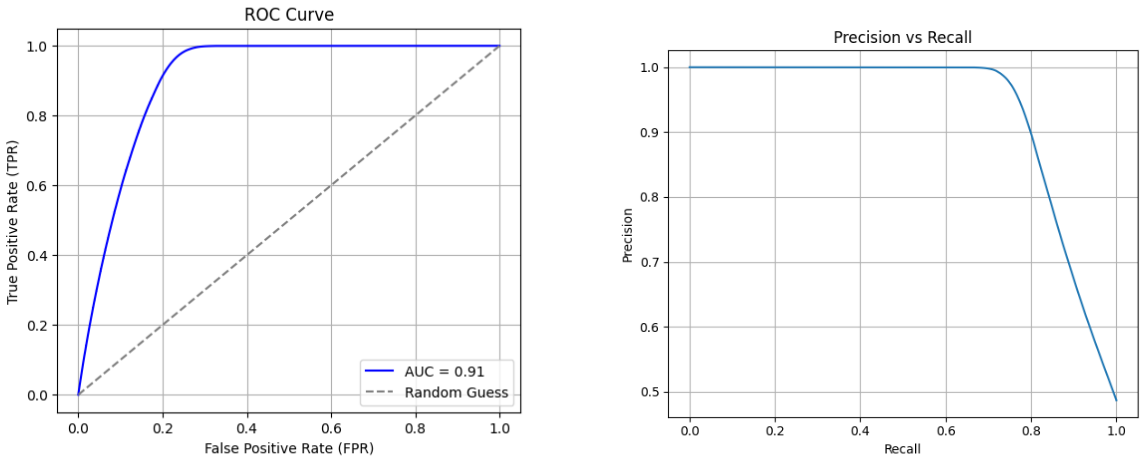


(a) Distribution of Predicted Probabilities (b) Confusion Matrix

Figure 8.2: Understanding Prediction distributions

In Figure 8.3a The ROC curve shows the trade-off between the True Positive Rate/Recall * and the False Positive Rate(FRP) † as these values are varied over different thresholds. The Recall tells us what fraction of the data classified as signal is correctly classified, while the false positive rate tells us what fraction of noise is incorrectly classified as signal. The Ideal curve here would touch the upper left corner since we would want our model to have perfect Recall and no false positives. The TPR can't ever be less than the FPR since that would mean that the model is actively misclassifying the data and you can fix that by simply inverting the predicted probabilities. The worst a model can perform is to be completely random. To measure how well a model is at classifying things we can calculate the area under the ROC curve the closer to 1 it is the better. The ROC can also inform us about the ideal threshold if we are looking to maximise the TPR or minimise the FPR. Similarly, the precision‡ recall curve in Figure 8.3b describes another method to find the ideal threshold. The difference between the 2 is that ROC is good for balancing both true positives and true negatives and is ideal for a balanced dataset while the precision recall curve is ideal for imbalanced datasets.

*TRP = TP/TP+FN
†FPR = FP/FP+TN
‡TP/TP+FP



(a) ROC

(b) Precision Recall Curve

Figure 8.3: Model Evaluation Metrics

9 Comparison of Results

9.1 GNN vs Coincidence

Finally, with both our GNN and CT triggers ready it is time to compare them and see which trigger performs better. Since we don't care what happens to the noise the perfect metric for us to use to figure out which of the 2 triggers is better is to use Recall. In Figure 9.1 we can see that the Recall vs Energy of the model that was trained on mixed data applied to a dataset it has never seen in comparison to the CT trigger note that the CT trigger performance is different from its performance in Figure 6.4 since in this graph we have used the trigger on a dataset with all the cuts that were mentioned in Chapter 8. Also at least in the case of tracks, it seems that the trigger's ability to correctly classify events is negatively correlated with energy in this data set. A possible explanation for this is that the cuts we applied more harshly affect the simulations with higher energies while the low-energy simulations are relatively less affected. This however is not the entire explanation when it comes to the GNN. While this downward trend probably has something to do with the cuts it's not quite so simple.

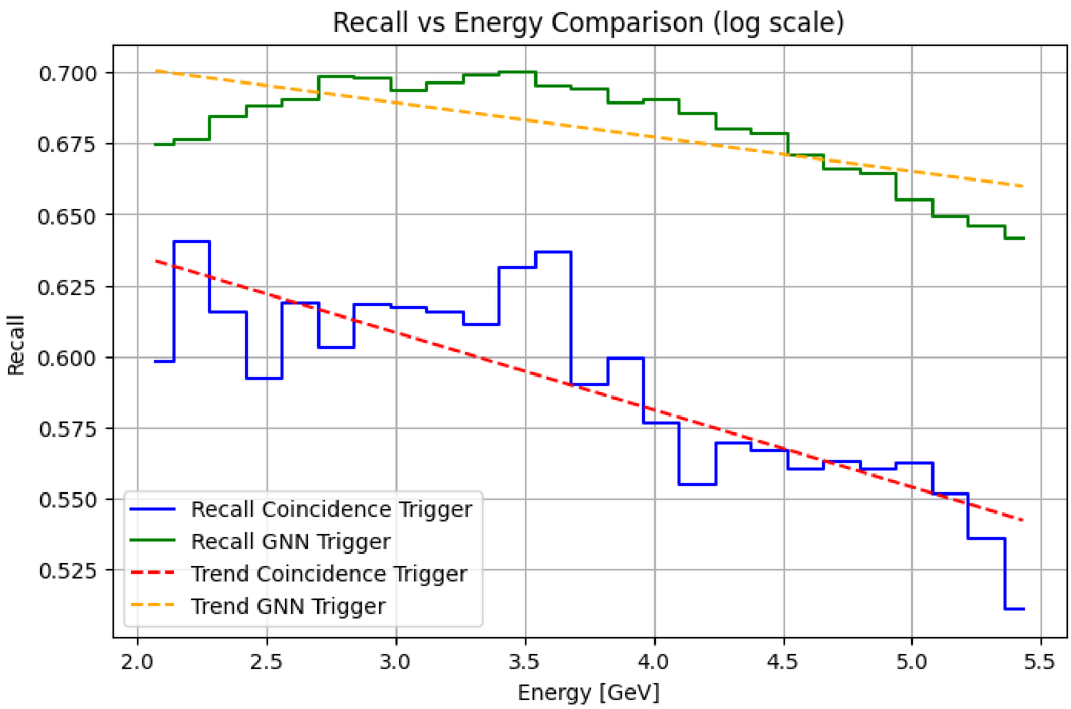
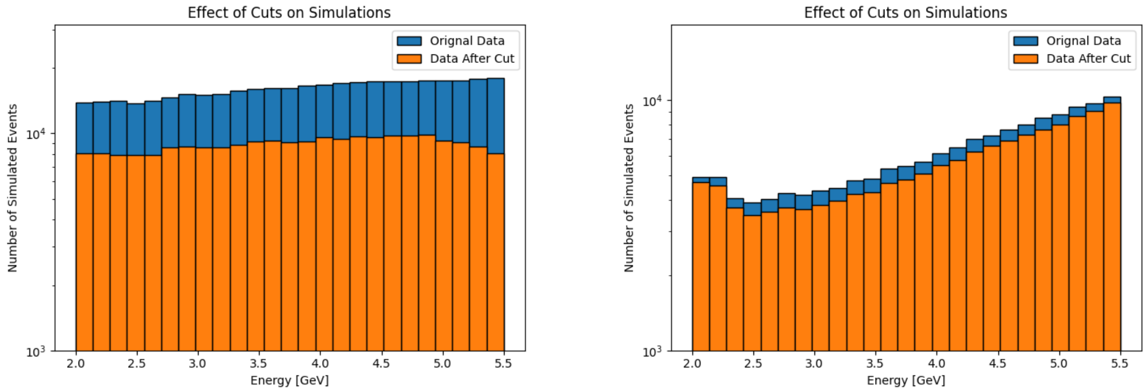


Figure 9.1: Model trained on Mixed data and CT triggering on cut data as described in Chapter 8

In Figure 9.3 and Figure 9.4 we can see that the GNN does show a positive trend with energy when the datasets are divided back into Cascades and Tracks. The downward trend for the GNN can probably be attributed to the fact that at higher energies ratio of cascades to track starts to skew towards tracks, we can see this from Figure 9.2 that at lower energies there is an excess of cascades while at higher energies the tracks seem to be roughly balanced with the number of cascades if not more than them. So based on this it's probably worth looking into the effects of datasets with just Cascades and Tracks on our triggers.



(a) Cascades (b) Tracks

Figure 9.2: Effect of Cuts on Energy Distribution

9.2 Cascades vs Tracks

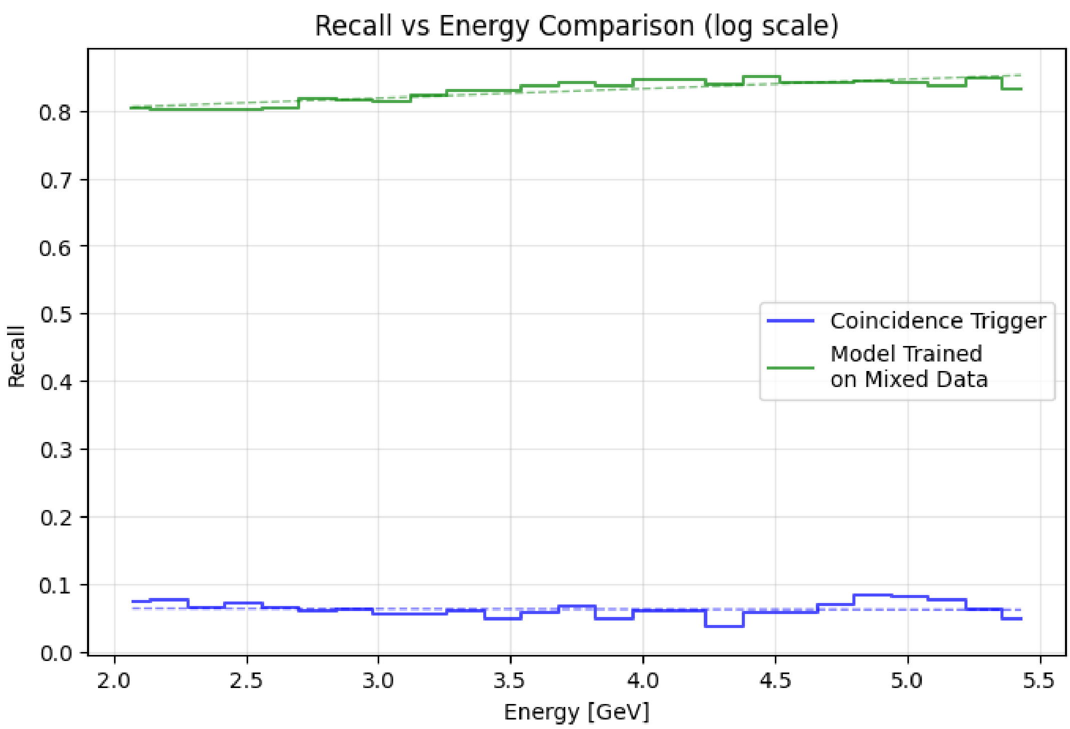


Figure 9.3: Cascade

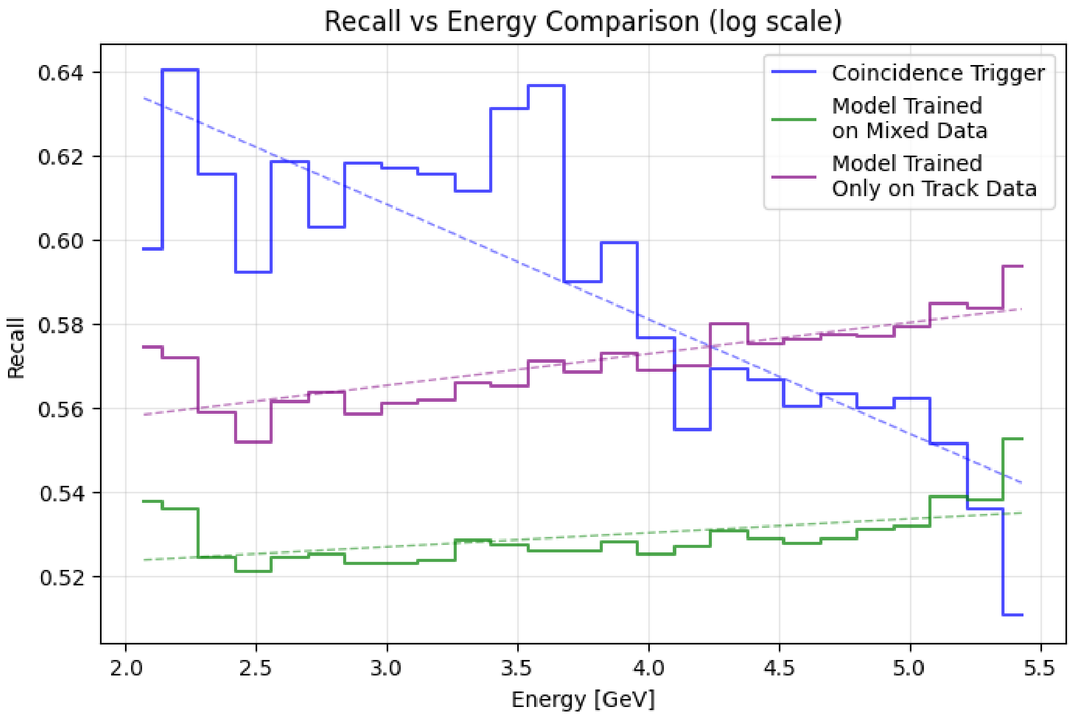


Figure 9.4: Understanding the effects of training datasets on the model using Tracks

Figure 9.1 shows us how our model behaves when given just the Cascade data and the contrast in the Recall values is stark. It looks like our model has learned to classify low-hit count cascades really well while the CT is struggling with the cuts. A possible explanation is that the cuts on the cascades, an event signature highly correlated with hits, adversely affected the performance of the coincidence trigger, perhaps because the majority of hits are deposited into a few DOMs due to the short-range nature of cascades and then the cuts get rid of these DOMs while leaving in traces of hits in neighbouring DOMs that the model was able to piece together and correctly classify events. However, this explanation seems a little too good for comfort and we would need further investigation to concretely say if this is truly as good a result as it appears.

Finally, we will consider Figure 9.4 in this case we see that the CT performs better than the Tracks, from Figure 9.2 we have seen that the effect of the cuts on tracks was negligible and this is also to be expected since Tracks in general do not have too many hits. The poor performance of the model on Tracks is however surprising hence we used a preliminary model, this model was created early on as a test and was not trained to completion i.e. model didn't have stable losses. However, it was only trained on Tracks so this allows us to compare at least qualitatively the difference in how different datasets may affect training, and even this partially trained model seems to be better at classifying tracks than the model trained on mixed data. This suggests that it is important to treat the two datasets differently while training, the reason for why this is the case is unclear but possibly the geographically distributed nature of tracks prevents the model from learning about tracks especially when it's focused on getting more localised cascades correct. The track trained model however indicates that at least it's theoretically possible for a model to have better performance on track.

10 Conclusions & Outlook

So to summarise throughout this thesis we have gone through many ideas, we talked about Ananke and Olympus in Chapter 4 and how they can be used to create simulations, and then we went ahead and created about 450,000 event simulations that were then injected with noise. In Chapter 6 we created a simple Coincidence Trigger and understood its behaviour and what were the optimal parameters for its performance.

In Chapter 7 we discussed the basics of Machine Learning and the Graph Neural Networks and then in Chapter 8 we discussed Graphnet and how the data was reorganised and the various cuts we put on the data and used it to finally implement a GNN trigger. We then saw how well it trained on the data it was given. Finally, in Chapter 9 we compared the CT trigger and GNN trigger and saw how they stacked up against each other, we saw that the GNN trigger is superior at classifying Cascades while struggling to classify Tracks and we discussed potential causes to this discrepancy.

So based on the work done so far many further tasks need to be done to fully understand the potential of GNN triggers in real-world applications, this includes understanding the discrepancy between the performance of CT and the GNN trigger on cascades. And in the same vein it is important to figure out how to train a model that will perform equally well on both cascades and tracks. This could take many shapes some ideas include moving the model from a binary classifier to a multi-classifier, explicitly balancing the dataset in terms of tracks and cascades along with balancing it with noise, and perhaps modifying the way the cuts were applied or the criteria for the cuts could help us see improvements.

Its also important to make sure that the model is capable of identifying the parts of the simulations that were cut out since this was never tested its entirely possible that the GNN trigger would reject any event with too many hits since that was not something it was trained on.

Another area of progress that can be made is the further development of the simulation software to be able to handle larger simulation workloads as the standard way to use the software does not scale well the number of events to be generated and a number of highly unstable workaround had to be employed to make the full use of the software.

Once a sufficiently good trigger has been developed it is also equally important to test it out in the real world and not just in simulations, and to do this we can implement our models on FPGAs* [69] which are a type of integrated circuit that are highly customisable and show great potential for integration with machine learning [70] and may be used to make fast triggers for P-ONE or other detectors that are currently under constructions or considering upgrades.

To conclude our GNN trigger has shown great potential to outperform standard triggers and with further tinkering even its current short comings can be overcome.

*Field-Programmable Gate Array

A HDF5 File Structure

Table A.1: Detector

Column Name	Description
string_location_x	x co-ordinate of the string
string_location_y	y co-ordinate of the string
string_location_z	z-co-ordinate of the string (always 0)
string_id	An integer identifying the string
module_id	An integer from $0-19$ identifying the DOM on the string
module_location_x	x co-ordinate of the DOM
module_location_y	y co-ordinate of the DOM
module_location_z	z co-ordinate of the DOM
module_radius	The radius of the DOM in m
pmt_orientation_x	x component of the PMTs normal vector
pmt_orientation_y	y component of the PMTs normal vector
pmt_orientation_z	z component of the PMTs normal vector
pmt_id	An integer from $0-15$ identifying the PMT in the DOM
pmt_efficiency	A float between $0-1$ that characterises the PMT response
pmt_area	PMT area in cm^2
pmt_noise_rate	Rate of noise generation in ns^{-1}
pmt_location_x	x co-ordinate of the PMT
pmt_location_y	y co-ordinate of the PMT
pmt_location_z	z co-ordinate of the PMT

Table A.2: Sources

Column Name	Description
record_id	A unique event identifying integer
location_x	x location of photon source
location_y	y location of photon source
location_z	z location of photon source
orientation_x	x component of the photon sources normal vector
orientation_y	y component of the photon sources normal vector
orientation_z	z component of the photon sources normal vector
time	start time of event in ns
number_of_photons	number of photons emitted by photon source
type	Integer value of the source type. Cherenkov sources are 0, and isotropic sources 1.

Table A.3: Hits

Column Name	Description
record_id	A unique event identifying integer
time	time of hit in ns
string_id	String id within cluster
module_id	DOM id within String
pmt_id	PMT id within DOM
type	Integer identifying if a hit is a Starting Track (0), Cascade(1), Realistic Track(2), Electrical Noise (20) or Bioluminescent Noise(21)

Table A.4: Records

Column Name	**Description**
record_id	A unique event identifying integer
type	Integer identifying if a hit is a Starting Track (0), Cascade(1), Realistic Track(2), Electrical Noise (20) or Bioluminescent Noise(21)
location_x	x co-ordinate of interaction
location_y	y co-ordinate of interaction
location_z	z co-ordinate of interaction
orientation_x	x component of the interaction normal vector
orientation_y	y component of the interaction normal vector
orientation_z	z component of the interaction normal vector
energy	Energy of the interaction in GeV
length	Length of the interaction in m
time	Start time in ns
duration	Event duration in ns
particle_id	Id of the interacting particle as used by the fennel software package [79] and defined in the Monte Carlo particle numbering scheme [80]. Optional for event records.

Table A.5: Features / Pules Map

Column Name	**Description**
event_no (index)	
time	time of hit in ns
string_id	An integer identifying the string
module_id	An integer from $0 - 19$ identifying the DOM on the string
pmt_id	An integer from $0 - 15$ identifying the PMT in the DOM
record_id	A unique event identifying integer (Can be used to link back to our simulation)
module_location_x	x coordinate of the module
module_location_y	y coordinate of the module
module_location_z	z coordinate of the module
pmt_orientation_x	x component of the PMT's orientation
pmt_orientation_y	y component of the PMT's orientation
pmt_orientation_z	z component of the PMT's orientation
pmt_location_x	x coordinate of the PMT
pmt_location_y	y coordinate of the PMT
pmt_location_z	z coordinate of the PMT

Table A.6: Truth

Column Name	**Description**
event_no (index)	A unique integer identifying the 100 ns chunks
location_x	x coordinate of the particle's location
location_y	y coordinate of the particle's location
location_z	z coordinate of the particle's location
orientation_x	x component of the particle's orientation
orientation_y	y component of the particle's orientation
orientation_z	z component of the particle's orientation
record_id	A unique event identifying integer (Can be used to link back to our simulation)
energy	Energy of the interaction in GeV
length	Length of the interaction in m
time	Start time in ns
particle_id	Id of the interacting particle as used by the fennel software package [79] and defined in the Monte Carlo particle numbering scheme [80]. Optional for event records.
duration	Event duration in ns
type	Either 0 if any of the hits with the associated event_no are due to neutrinos else 1

B Detector Fact Sheets

B.1 Baikal

Table B.1: Baikal [71] [72]

Column Name	Description
Absorption length in water	1m
OM spacing	15m
OMs per string	36
Number of clusters	3
Number of strings per cluster	8
PMT Diameter	250 mm
Scattering length	30-50m
String depth	1366m
String length	525m
String spacing	60m

B.2 IceCube

Table B.2: IceCube[73]

Column Name	Description
DOM spacing	17/7m
DOMs per string	60
Number of PMTs per module	1
Number of strings	86
PMT Diameter	25cm
String depth	2500m
String length	1000m
String spacing	125/70m

B.3 KM3Net

Table B.3: ORCA and ARCA[45, 74–76]

Column Name	ORCA	ARCA
Attenuation length	35m	35m
Detector Depth	2450	3500
DOM spacing	9m	36m
DOMs per string	18	18
Number of PMTs per module	31	31
Number of strings	115	115
PMT Diameter	3-inch	3-inch
String depth	200m	700m
String spacing	20m	95m

B.4 TRIDENT

Table B.4: TRIDENT [50]

Column Name	Description
Absorption length	27m
DOM spacing	30m
DOMs per string	20
Number of strings	1211
String depth	3.5km
String length	27m
String spacing	70-110m

B.5 P-ONE

Table B.5: P-ONE[77]

Column Name	Description
Attenuation length	20-40m
DOM spacing	50m
DOMs per string	20
Number of PMTs per module	16/10
Number of strings per cluster	10
String depth	2660m
String length	1000m
String spacing	80m

References

[1] Alan A. Watson. The discovery of cherenkov radiation and its use in the detection of extensive air showers. *Nuclear Physics B - Proceedings Supplements*, 212–213: 13–19, March 2011. ISSN 0920-5632. doi: 10.1016/j.nuclphysbps.2011.03.003. URL http://dx.doi.org/10.1016/j.nuclphysbps.2011.03.003.

[2] Hans Niederhausen. Recent icecube measurements using high energy neutrinos, 2019. URL https://arxiv.org/abs/1909.12182.

[3] Matei Dan Ciobotaru. *Characterizing, managing and monitoring the networks for the ATLAS data acquisition system.* PhD thesis, Bucharest, Tech. U., 2007.

[4] The KM3NeT Collaboration, S. Aiello, F. Ameli, et al. Km3net front-end and readout electronics system: hardware, firmware and software, 2019. URL https://arxiv.org/abs/1907.06453.

[5] Thomas K. Gaisser, Todor Stanev, and Serap Tilav. Cosmic ray energy spectrum from measurements of air showers, 2013. URL https://arxiv.org/abs/1303.3565.

[6] S. Minardi, R. J. Harris, and L. Labadie. Astrophotonics: astronomy and modern optics. *The Astronomy and Astrophysics Review*, 29(1):6, 2021. doi: 10.1007/s00159-021-00134-7. URL https://doi.org/10.1007/s00159-021-00134-7.

[7] Andrea Santangelo, Rosalia Madonia, and Santina Piraino. *A Chronological History of X-Ray Astronomy Missions*, pages 1–68. Springer Nature Singapore, 2023. ISBN 9789811645440. doi: 10.1007/978-981-16-4544-0_147-1. URL http://dx.doi.org/10.1007/978-981-16-4544-0_147-1.

[8] A. S. Eddington. The internal constitution of the stars. *The Scientific Monthly*, 11 (4):297–303, 1920. ISSN 0096-3771. URL http://www.jstor.org/stable/6491.

[9] Bruce A. Remington, R. Paul Drake, Hideaki Takabe, et al. A review of astrophysics experiments on intense lasers. *Physics of Plasmas*, 7(5):1641–1652, May 2000. ISSN 1070-664X. doi: 10.1063/1.874046. URL https://doi.org/10.1063/1.874046.

[10] Aristotle Socrates and Lorenzo Sironi. Photon feedback: Screening and the eddington limit. *The Astrophysical Journal*, 772(2):L21, Jul 2013. ISSN 2041-8213. doi: 10.1088/2041-8205/772/2/L21. URL http://dx.doi.org/10.1088/2041-8205/772/2/L21.

[11] Robert J. Gould and Gerald Schréder. Opacity of the universe to high-energy photons. *Phys. Rev. Lett.*, 16(6):252–254, Feb 1966. doi: 10.1103/PhysRevLett.16.252. URL https://link.aps.org/doi/10.1103/PhysRevLett.16.252.

[12] G. T. Zatsepin and V. A. Kuz'min. Upper limit of the spectrum of cosmic rays. *Soviet Journal of Experimental and Theoretical Physics Letters*, 4:78, Aug 1966.

[13] B. P. Abbott, R. Abbott, T. D. Abbott, et al. Tests of general relativity with gw150914. *Physical Review Letters*, 116(22), May 2016. ISSN 1079-7114. doi: 10.1103/physrevlett.116.221101. URL http://dx.doi.org/10.1103/PhysRevLett.116.221101.

[14] Raymond Davis, Don S. Harmer, and Kenneth C. Hoffman. Search for neutrinos from the sun. *Phys. Rev. Lett.*, 20:1205–1209, May 1968. doi: 10.1103/PhysRevLett.20.1205. URL https://link.aps.org/doi/10.1103/PhysRevLett.20.1205.

[15] K. Scholberg. The supernova early warning system. *Astronomische Nachrichten*, 329(3):337–339, March 2008. ISSN 1521-3994. doi: 10.1002/asna.200710934. URL http://dx.doi.org/10.1002/asna.200710934.

[16] Pietro Antonioli, Richard Tresch Fienberg, Fabrice Fleurot, et al. Snews: the supernova early warning system. *New Journal of Physics*, 6:114, September 2004. ISSN 1367-2630. doi: 10.1088/1367-2630/6/1/114. URL http://dx.doi.org/10.1088/1367-2630/6/1/114.

[17] ANTARES Collaboration. A deep sea telescope for high energy neutrinos. *arXiv preprint*, 1999. URL https://arxiv.org/abs/astro-ph/9907432.

[18] S. Bilenky. Neutrino oscillations: From a historical perspective to the present status. *Nuclear Physics B*, 908:2–13, 2016. ISSN 0550-3213. doi: https://doi.org/10.1016/j.nuclphysb.2016.01.025. URL https://www.sciencedirect.com/science/article/pii/S0550321316000353. Neutrino Oscillations: Celebrating the Nobel Prize in Physics 2015.

[19] S.M. Bilenky and B. Pontecorvo. Lepton mixing and neutrino oscillations. *Physics Reports*, 41(4):225–261, 1978. ISSN 0370-1573. doi: https://doi.org/10.1016/0370-1573(78)90095-9. URL https://www.sciencedirect.com/science/article/pii/0370157378900959.

[20] MARCO DREWES. The phenomenology of right handed neutrinos. *International Journal of Modern Physics E*, 22(08):1330019, August 2013. ISSN 1793-6608. doi: 10.1142/s0218301313300191. URL http://dx.doi.org/10.1142/S0218301313300191.

[21] TIM GERSHON. Overview of the cabibbo–kobayashi–maskawa matrix†. *Pramana*, 79(5):1091–1108, November 2012. ISSN 0973-7111. doi: 10.1007/s12043-012-0418-y. URL http://dx.doi.org/10.1007/s12043-012-0418-y.

[22] *Il Nuovo Cimento C*, 37(3):145–154, July 2014. ISSN 03905551, 03905551. doi: 10.1393/ncc/i2014-11774-6. URL https://doi.org/10.1393/ncc/i2014-11774-6.

[23] Y. Fukuda, T. Hayakawa, E. Ichihara, et al. Evidence for oscillation of atmospheric neutrinos. *Physical Review Letters*, 81(8):1562–1567, August 1998. ISSN 1079-7114. doi: 10.1103/physrevlett.81.1562. URL http://dx.doi.org/10.1103/PhysRevLett.81.1562.

[24] Alessio Maiezza, Goran Senjanović, and Juan Carlos Vasquez. Higgs sector of the minimal left-right symmetric theory. *Physical Review D*, 95(9), May 2017. ISSN 2470-0029. doi: 10.1103/physrevd.95.095004. URL http://dx.doi.org/10.1103/PhysRevD.95.095004.

[25] Massimiliano Lattanzi, Roberto A Lineros, and Marco Taoso. Connecting neutrino physics with dark matter. *New Journal of Physics*, 16(12):125012, December 2014. ISSN 1367-2630. doi: 10.1088/1367-2630/16/12/125012. URL http://dx.doi.org/10.1088/1367-2630/16/12/125012.

[26] Patrick deNiverville, Maxim Pospelov, and Adam Ritz. Observing a light dark matter beam with neutrino experiments. *Physical Review D*, 84(7), October 2011. ISSN 1550-2368. doi: 10.1103/physrevd.84.075020. URL http://dx.doi.org/10.1103/PhysRevD.84.075020.

[27] R. Abbasi, Y. Abdou, T. Abu-Zayyad, et al. Multiyear search for dark matter annihilations in the sun with the amanda-ii and icecube detectors. *Physical Review D*, 85(4), February 2012. ISSN 1550-2368. doi: 10.1103/physrevd.85.042002. URL http://dx.doi.org/10.1103/PhysRevD.85.042002.

[28] Lars Bergström, Joakim Edsjö, and Paolo Gondolo. Indirect detection of dark matter in km-size neutrino telescopes. *Phys. Rev. D*, 58:103519, Oct 1998. doi: 10.1103/PhysRevD.58.103519. URL https://link.aps.org/doi/10.1103/PhysRevD.58.103519.

[29] C. MARIANI. Review of reactor neutrino oscillation experiments. *Modern Physics Letters A*, 27(08):1230010, March 2012. ISSN 1793-6632. doi: 10.1142/s0217732312300108. URL http://dx.doi.org/10.1142/S0217732312300108.

[30] Carlo Bemporad, Giorgio Gratta, and Petr Vogel. Reactor-based neutrino oscillation experiments. *Reviews of Modern Physics*, 74(2):297–328, March 2002. ISSN 1539-0756. doi: 10.1103/revmodphys.74.297. URL http://dx.doi.org/10.1103/RevModPhys.74.297.

[31] Antoine Kouchner. Next-generation atmospheric neutrino experiments. *Physics of the Dark Universe*, 4:60–74, 2014. ISSN 2212-6864. doi: https://doi.org/10.1016/j.dark.2014.09.001. URL https://www.sciencedirect.com/science/article/pii/S2212686414000259. DARK TAUP2013.

[32] Francis Halzen and Dan Hooper. High-energy neutrino astronomy: the cosmic ray connection. *Reports on Progress in Physics*, 65(7):1025–1078, June 2002. ISSN 0034-4885. doi: 10.1088/0034-4885/65/7/201. URL http://dx.doi.org/10.1088/0034-4885/65/7/201.

[33] Thomas K. Gaisser, Francis Halzen, and Todor Stanev. Particle astrophysics with high energy neutrinos. *Physics Reports*, 258(3):173–236, 1995. ISSN 0370-1573. doi: https://doi.org/10.1016/0370-1573(95)00003-Y. URL https://www.sciencedirect.com/science/article/pii/037015739500003Y.

[34] John G Learned and Sandip Pakvasa. Detecting oscillations at pev energies. *Astroparticle Physics*, 3(3):267–274, 1995. ISSN 0927-6505. doi: https://doi.org/10.1016/0927-6505(94)00043-3. URL https://www.sciencedirect.com/science/article/pii/0927650594000433.

[35] Michael E. Peskin and Daniel V. Schroeder. *An Introduction to Quantum Field Theory*. Westview Press, Boulder, CO, 1995. ISBN 978-0-201-50397-5.

[36] Lucio Cerrito. *Cherenkov and Transition Radiation: Detectors for PID and Neutrinos*, pages 187–196. Springer International Publishing, Cham, 2017. ISBN 978-3-319-53181-6. doi: 10.1007/978-3-319-53181-6_11. URL https://doi.org/10.1007/978-3-319-53181-6_11.

[37] John David Jackson. *Classical electrodynamics*. Wiley, New York, NY, 3rd ed. edition, 1999. ISBN 9780471309321. URL http://cdsweb.cern.ch/record/490457.

[38] M. G. Aartsen, M. Ackermann, J. Adams, et al. Search for sources of astrophysical neutrinos using seven years of icecube cascade events. *The Astrophysical Journal*, 886(1):12, nov 2019. doi: 10.3847/1538-4357/ab4ae2. URL https://dx.doi.org/10.3847/1538-4357/ab4ae2.

[39] M G Aartsen, R Abbasi, M Ackermann, et al. Energy reconstruction methods in the icecube neutrino telescope. *Journal of Instrumentation*, 9(03):P03009–P03009, March 2014. ISSN 1748-0221. doi: 10.1088/1748-0221/9/03/p03009. URL http://dx.doi.org/10.1088/1748-0221/9/03/P03009.

[40] Q. Dorosti Hasankiadeh, O. Kavatsyuk, H. Löhner, et al. Performance of photosensors for km3net. *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment*, 725:158–161, 2013. ISSN 0168-9002. doi: https://doi.org/10.1016/j.nima.2012.11.154. URL https://www.sciencedirect.com/science/article/pii/S016890021201501X. VLV T 11, Erlangen, Germany, 12 - 14 October, 2011.

[41] F. Robben. Noise in the measurement of light withphotomultipliers. *Appl. Opt.*, 10 (4):776–796, Apr 1971. doi: 10.1364/AO.10.000776. URL https://opg.optica.org/ao/abstract.cfm?URI=ao-10-4-776.

[42] Christian Spannfellner. Design of the Pacific Ocean Neutrino Experiment‘s First Detector Line. *PoS*, ICRC2023:1219, 2023. doi: 10.22323/1.444.1219.

[43] S. Aiello, A. Albert, M. Alshamsi, et al. The km3net multi-pmt optical module. *Journal of Instrumentation*, 17(07):P07038, jul 2022. doi: 10.1088/1748-0221/17/07/P07038. URL https://dx.doi.org/10.1088/1748-0221/17/07/P07038.

[44] Kilian Holzapfel. *Bioluminescence in the Pacific Ocean Neutrino Experiment: Shedding Light on the Deep Sea*. PhD thesis, Technische Universität München, 2023. URL https://mediatum.ub.tum.de/1713703.

[45] P. Bagley and others. KM3NeT: Technical Design Report for a Deep-Sea Research Infrastructure in the Mediterranean Sea Incorporating a Very Large Volume Neutrino Telescope, 2009. Technical Design Report.

[46] M. I. Latz, T. M. Frank, and J. F. Case. Spectral composition of bioluminescence of epipelagic organisms from the sargasso sea. *Marine Biology*, 98(3):441–446, 1988. doi: 10.1007/BF00391120. URL `https://doi.org/10.1007/BF00391120`.

[47] P. J. Herring and Malcolm Roy Clarke. The spectral characteristics of luminous marine organisms. *Proceedings of the Royal Society of London. Series B. Biological Sciences*, 220(1219):183–217, 1983. doi: 10.1098/rspb.1983.0095. URL `https://royalsocietypublishing.org/doi/abs/10.1098/rspb.1983.0095`.

[48] M. Boehmer, J. Bosma, D. Brussow, et al. Straw (strings for absorption length in water): pathfinder for a neutrino telescope in the deep pacific ocean. *Journal of Instrumentation*, 14(02):P02013, feb 2019. doi: 10.1088/1748-0221/14/02/P02013. URL `https://dx.doi.org/10.1088/1748-0221/14/02/P02013`.

[49] F. Kulcsár, D. Teherani, and H. Altmann. Study of the spectrum of cherenkov light. *Journal of radioanalytical chemistry*, 68(1):161–168, 1982. doi: 10.1007/BF02517618. URL `https://doi.org/10.1007/BF02517618`.

[50] Z. P. Ye, F. Hu, W. Tian, et al. A multi-cubic-kilometre neutrino telescope in the western pacific ocean, 2024. URL `https://arxiv.org/abs/2207.04519`.

[51] A. D. Avrorin, A. V. Avrorin, V. M. Aynutdinov, et al. High-energy neutrino astronomy and the baikal-gvd neutrino telescope. *Physics of Atomic Nuclei*, 84(4): 513–518, 2021. doi: 10.1134/S1063778821040062. URL `https://doi.org/10.1134/S1063778821040062`.

[52] Jean Pierre Twagirayezu, Hans Niederhausen, Stephen Sclafani, et al. Performance of the Pacific Ocean Neutrino Experiment (P-ONE). *PoS*, ICRC2023:1175, 2023. doi: 10.22323/1.444.1175.

[53] R. Abbasi, Y. Abdou, T. Abu-Zayyad, et al. The design and performance of icecube deepcore. *Astroparticle Physics*, 35(10):615–624, 2012. ISSN 0927-6505. doi: https://doi.org/10.1016/j.astropartphys.2012.01.004. URL `https://www.sciencedirect.com/science/article/pii/S0927650512000254`.

[54] M. G. Aartsen, K. Abraham, M. Ackermann, et al. Evidence for astrophysical muon neutrinos from the northern sky with icecube. *Physical Review Letters*, 115 (8), August 2015. ISSN 1079-7114. doi: 10.1103/physrevlett.115.081102. URL `http://dx.doi.org/10.1103/PhysRevLett.115.081102`.

[55] Stephan Meighen-Berger, Li Ruohan, and Golo Wimmer. Bioluminescence modeling for deep sea experiments, 2021. URL `https://arxiv.org/abs/2103.03816`.

[56] Felix J. Yu, Nicholas Kamp, and Carlos A. Argüelles. Learning efficient representations of neutrino telescope events, 2024. URL `https://arxiv.org/abs/2410.13148`.

[57] Tom M. Mitchell. *Machine Learning*. McGraw-Hill, New York, NY, USA, 1997.

[58] Johannes Lederer. Activation functions in artificial neural networks: A systematic overview, 2021. URL https://arxiv.org/abs/2101.09957.

[59] Derya Soydaner. A comparison of optimization algorithms for deep learning. *International Journal of Pattern Recognition and Artificial Intelligence*, 34(13): 2052013, April 2020. ISSN 1793-6381. doi: 10.1142/s0218001420520138. URL http://dx.doi.org/10.1142/S0218001420520138.

[60] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017. URL https://arxiv.org/abs/1412.6980.

[61] Hossein Gholamalinezhad and Hossein Khosravi. Pooling methods in deep neural networks, a review, 2020. URL https://arxiv.org/abs/2009.07485.

[62] Ibomoiye Domor Mienye, Theo G. Swart, and George Obaido. Recurrent neural networks: A comprehensive review of architectures, variants, and applications. *Information*, 15(9), 2024. ISSN 2078-2489. doi: 10.3390/info15090517. URL https://www.mdpi.com/2078-2489/15/9/517.

[63] Jie Zhou, Ganqu Cui, Shengding Hu, et al. Graph neural networks: A review of methods and applications. *AI Open*, 1:57–81, 2020. ISSN 2666-6510. doi: https://doi.org/10.1016/j.aiopen.2021.01.001. URL https://www.sciencedirect.com/science/article/pii/S2666651021000012.

[64] Bharti Khemani, Shruti Patil, Ketan Kotecha, et al. A review of graph neural networks: concepts, architectures, techniques, challenges, datasets, applications, and future directions. *Journal of Big Data*, 11(1):18, 2024. doi: 10.1186/s40537-023-00876-4. URL https://doi.org/10.1186/s40537-023-00876-4.

[65] Xue Li and Yuanzhi Cheng. Understanding the message passing in graph neural networks via power iteration clustering. *Neural Networks*, 140:130–135, 2021. ISSN 0893-6080. doi: https://doi.org/10.1016/j.neunet.2021.02.025. URL https://www.sciencedirect.com/science/article/pii/S0893608021000757.

[66] Kashvi Taunk, Sanjukta De, Srishti Verma, et al. A brief review of nearest neighbor algorithm for learning and classification. In *2019 International Conference on Intelligent Computing and Control Systems (ICCS)*, pages 1255–1260, 2019. doi: https://doi.org/10.1109/ICCS45141.2019.9065747.

[67] Margherita Grandini, Enrico Bagli, and Giorgio Visani. Metrics for multi-class classification: an overview, 2020. URL https://arxiv.org/abs/2008.05756.

[68] Oona Rainio, Jarmo Teuho, and Riku Klén. Evaluation metrics and statistical tests for machine learning. *Scientific Reports*, 14(1):6086, 2024. doi: 10.1038/s41598-024-56706-x. URL https://doi.org/10.1038/s41598-024-56706-x.

[69] Hartmut F.-W. Sadrozinski and J. Wu. *Applications of Field-Programmable Gate Arrays in Scientific Research*. CRC Press, 1st edition, 2010. doi: 10.1201/b10480.

[70] Hongzheng Chen, Jiahao Zhang, Yixiao Du, et al. Understanding the potential of fpga-based spatial acceleration for large language model inference. *ACM Transactions on Reconfigurable Technology and Systems*, 18(1):1–29, December 2024. ISSN 1936-7414. doi: 10.1145/3656177. URL `http://dx.doi.org/10.1145/3656177`.

[71] I.A Belolaptikov, L.B Bezrukov, B.A Borisovets, et al. The baikal underwater neutrino telescope: Design, performance, and first results. *Astroparticle Physics*, 7(3):263–282, 1997. ISSN 0927-6505. doi: https://doi.org/10.1016/S0927-6505(97)00022-4. URL `https://www.sciencedirect.com/science/article/pii/S0927650597000224`.

[72] Avrorin, A.D., Avrorin, A.V., Aynutdinov, V.M., et al. Baikal-gvd: status and prospects. *EPJ Web Conf.*, 191:01006, 2018. doi: 10.1051/epjconf/201819101006. URL `https://doi.org/10.1051/epjconf/201819101006`.

[73] Spencer R. Klein. Icecube: A cubic kilometer radiation detector. *IEEE Transactions on Nuclear Science*, 56(3):1141–1147, June 2009. ISSN 0018-9499. doi: 10.1109/tns.2009.2015300. URL `http://dx.doi.org/10.1109/TNS.2009.2015300`.

[74] A. Sinopoulou, R. Coniglione, R. Muller, et al. Atmospheric neutrinos with the first detection units of km3net/arca. *Journal of Instrumentation*, 16(11):C11015, November 2021. ISSN 1748-0221. doi: 10.1088/1748-0221/16/11/c11015. URL `http://dx.doi.org/10.1088/1748-0221/16/11/C11015`.

[75] S Adrián-Martínez, M Ageron, F Aharonian, et al. Letter of intent for km3net 2.0. *Journal of Physics G: Nuclear and Particle Physics*, 43(8):084001, jun 2016. doi: 10.1088/0954-3899/43/8/084001. URL `https://dx.doi.org/10.1088/0954-3899/43/8/084001`.

[76] G. Riccobene, A. Capone, S. Aiello, et al. Deep seawater inherent optical properties in the southern ionian sea. *Astroparticle Physics*, 27(1):1–9, 2007. ISSN 0927-6505. doi: https://doi.org/10.1016/j.astropartphys.2006.08.006. URL `https://www.sciencedirect.com/science/article/pii/S092765050600123X`.

[77] Paweł Malecki. Pacific ocean neutrino experiment. *Universe*, 10(2), 2024. ISSN 2218-1997. doi: 10.3390/universe10020053. URL `https://www.mdpi.com/2218-1997/10/2/53`.